CISSP 600

Security Assessment and Testing



AL NAFI,
A company with a focus on education, wellbeing and renewable energy.

© 2018 Al-Nafi. All Rights Reserved.

Forty Hadith on the importance of Knowledge, learning and Teaching.

© 2018 Al-Nafi. All Rights Reserved.

Hadith # 13 Allah's Path

Anas (may Allah be pleased with him) said: The Messenger of Allah (Peace Be upon Him) Said:

"He who leave his home in order to seek knowledge, he is in Allah's path until he returns [to his home]."

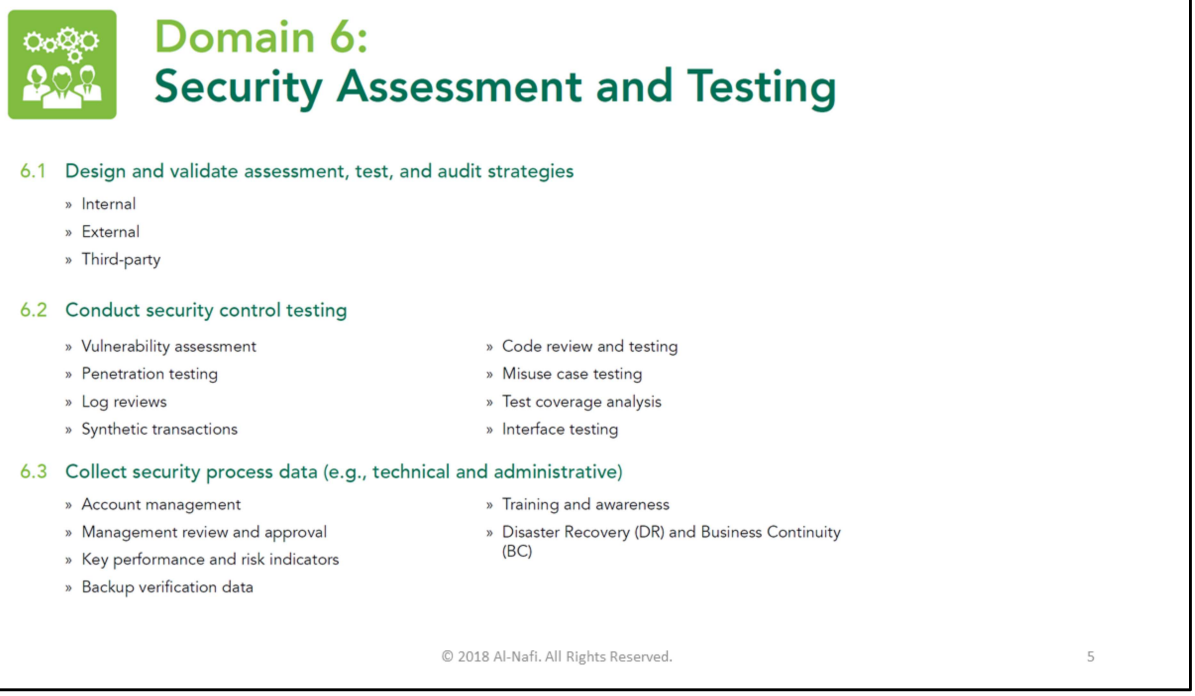
(at-Tirmidhi, Sunan; An-Nawawi, Riyad as-Salihin)

© 2018 Al-Nafi. All Rights Reserved.

How Nafi Members Study!

1. Please subscribe to our YouTube channel https://www.youtube.com/channel/UC2yAW4Oq27r1yuRE8ePKRvA
2. Follow us on Facebook https://www.facebook.com/info.alnafi/
3. Follow us on Twitter https://twitter.com/nafiPakistan
4. All Nafi members MUST study on the portal https://alnafi.com/login/ and connect using your Nafi member username and password. If you have problems connecting then please contact us via info@alnafi.com
5. To ask questions as it relates to studies please join our group https://www.facebook.com/groups/alnafi/
6. Once on the portal they can follow their classes by:
    - watching videos
    - asking questions
    - attempting quizzes and flash cards
    - studying official Nafi notes
    - keep track of their studies long with many more features

© 2018 Al-Nafi. All Rights Reserved.



Security testing and assessment are activities that assist an organization in managing risk, developing applications, managing systems, and utilizing services. To be successful in mitigating risks,
organizations must develop competencies that align with business needs related to assessing, validating, testing, and auditing systems and applications that support business objectives and goals.



Domain 6:
Security Assessment and Testing

6.4 Analyze test output and generate report

6.5 Conduct or facilitate security audits

- Internal
- External
- Third-party

© 2018 Al-Nafi. All Rights Reserved.

Design and Validate Assessment, Test, and Audit Strategies

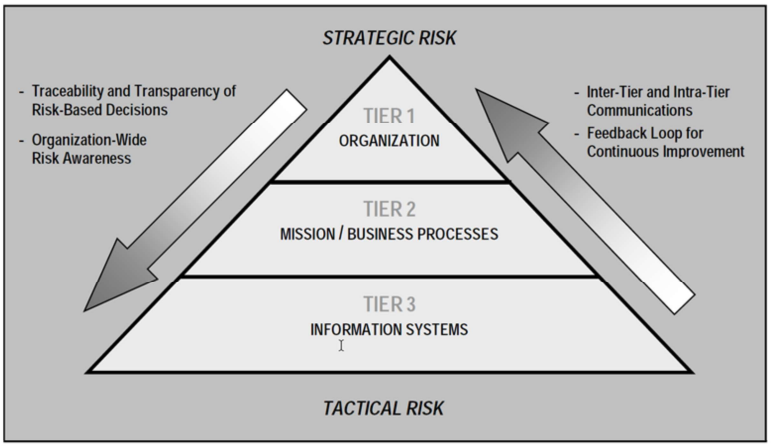
Security and Privacy Controls for Federal Information Systems and Organizations

Need to download and read this document:

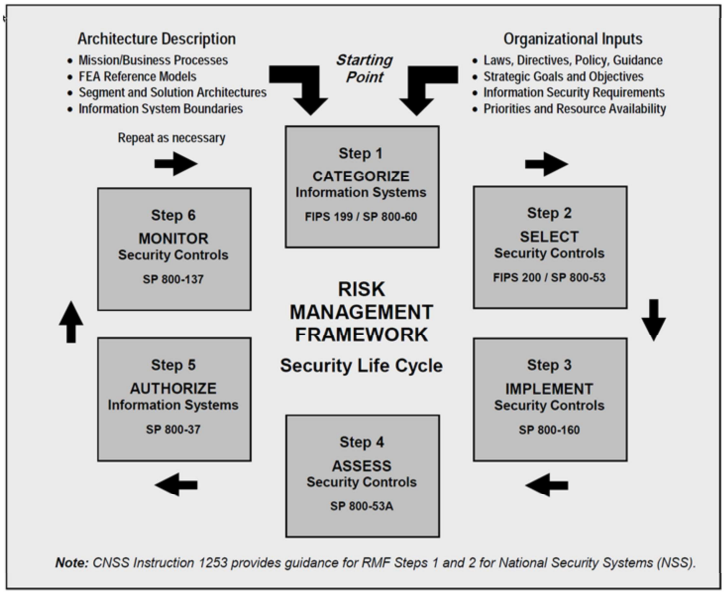
https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r4.pdf

© 2018 Al-Nafi. All Rights Reserved.

Design and Validate Assessment, Test, and Audit Strategies



© 2018 Al-Nafi. All Rights Reserved.



© 2018 Al-Nafi. All Rights Reserved.

Internal Actors/Threats

Insider Threat Study: Illicit Cyber Activity Involving Fraud in the U.S. Financial Services Sector

Adam Cummings
Todd Lewellen
David McIntire
Andrew P. Moore
Randall Trzeciak

July 2012

SPECIAL REPORT
CMU/SEI-2012-SR-004

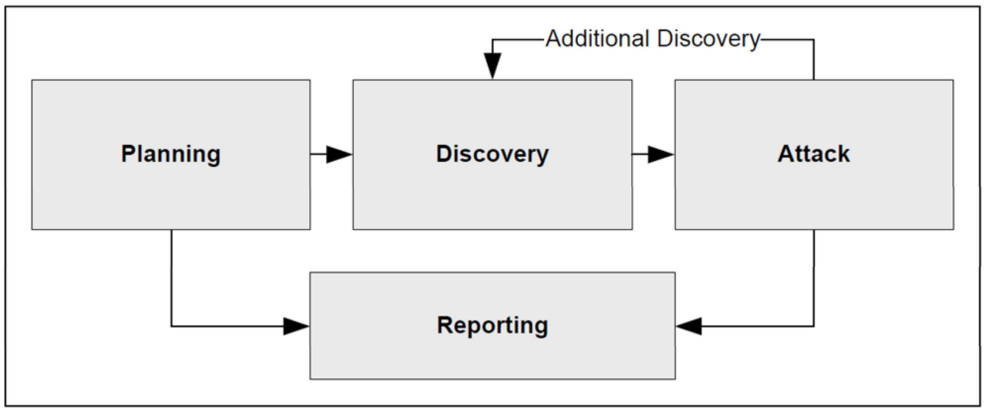
CERT Program
http://www.sei.cmu.edu



© 2018 Al-Nafi. All Rights Reserved.

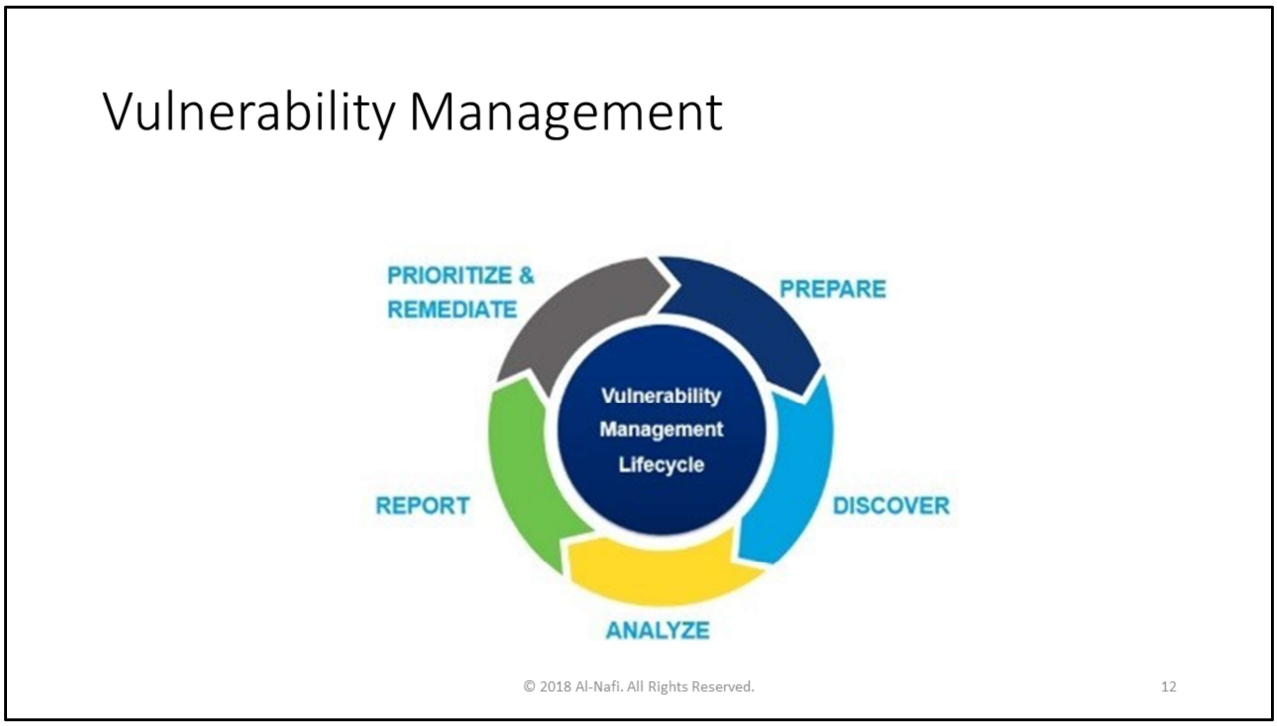
Please download and study this study
https://resources.sei.cmu.edu/asset_files/SpecialReport/2012_003_001_28137.pdf

Technical Guide to Information Security Testing and Assessment **Special Publication 800-115**



© 2018 Al-Nafi. All Rights Reserved.

https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf please download and read this document further.



12.6 Technical vulnerability management
Objective: To prevent exploitation of technical vulnerabilities.

12.6.1 Management of technical vulnerabilities
Control
Information about technical vulnerabilities of information systems being used should be obtained in a timely fashion, the organization's exposure to such vulnerabilities evaluated and appropriate measures taken to address the associated risk.

Implementation guidance
A current and complete inventory of assets (see Clause 8) is a prerequisite for effective technical vulnerability management. Specific information needed to support technical vulnerability management includes the software vendor, version numbers, current state of deployment (e.g. what software is installed on what systems) and the person(s) within the organization responsible for the software.

Appropriate and timely action should be taken in response to the identification of

potential technical vulnerabilities. The following guidance should be followed to establish an effective management process for technical vulnerabilities:
a) the organization should define and establish the roles and responsibilities associated with technical vulnerability management, including vulnerability monitoring, vulnerability risk assessment,
patching, asset tracking and any coordination responsibilities required;
b) information resources that will be used to identify relevant technical vulnerabilities and to maintain awareness about them should be identified for software and other technology (based on the asset inventory list, see 8.1.1); these information resources should be updated based on changes in the inventory or when other new or useful resources are found;
c) a timeline should be defined to react to notifications of potentially relevant technical vulnerabilities;
d) once a potential technical vulnerability has been identified, the organization should identify the associated risks and the actions to be taken; such action could involve patching of vulnerable
systems or applying other controls;
e) depending on how urgently a technical vulnerability needs to be addressed, the action taken should be carried out according to the controls related to change management (see 12.1.2) or by following information security incident response procedures (see 16.1.5);
f) if a patch is available from a legitimate source, the risks associated with installing the patch should be assessed (the risks posed by the vulnerability should be compared with the risk of installing the patch);
g) patches should be tested and evaluated before they are installed to ensure they are effective and do not result in side effects that cannot be tolerated; if no patch is available, other controls should be considered, such as:
1) turning off services or capabilities related to the vulnerability;
2) adapting or adding access controls, e.g. firewalls, at network borders (see 13.1);
3) increased monitoring to detect actual attacks;
4) raising awareness of the vulnerability;
h) an audit log should be kept for all procedures undertaken;
i) the technical vulnerability management process should be regularly monitored and evaluated in order to ensure its effectiveness and efficiency;
j) systems at high risk should be addressed first;
k) an effective technical vulnerability management process should be aligned with incident management activities, to communicate data on vulnerabilities to the incident response function
and provide technical procedures to be carried out should an incident occur;
l) define a procedure to address the situation where a vulnerability has been identified but there is no suitable countermeasure. In this situation, the organization

should evaluate risks relating to the known vulnerability and define appropriate detective and corrective actions.

Other information
Technical vulnerability management can be viewed as a sub-function of change management and as such can take advantage of the change management processes and procedures (see 12.1.2 and 14.2.2). Vendors are often under significant pressure to release patches as soon as possible. Therefore, there is a possibility that a patch does not address the problem adequately and has negative side effects. Also, in some cases, uninstalling a patch cannot be easily achieved once the patch has been applied.

Pentesting or offensive hacking or ethical hacking



© 2018 Al-Nafi. All Rights Reserved.

Pentesting steps



© 2018 Al-Nafi. All Rights Reserved.

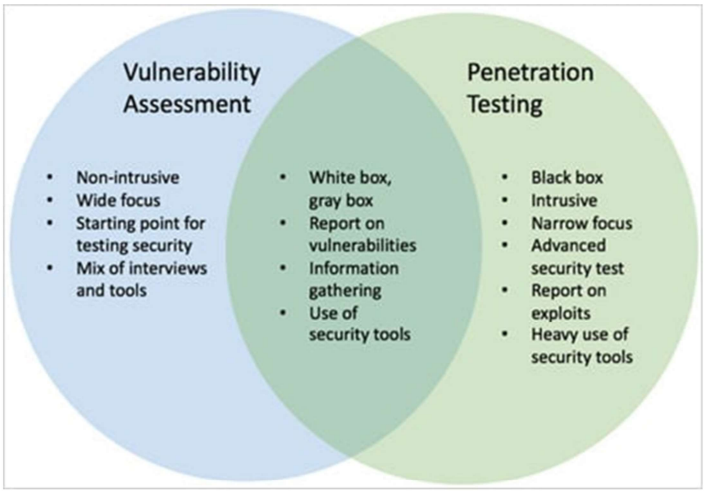
B.29 Pentest and vulnerability assessment

Information descriptor	Meaning or purpose
Measure ID	Organization-defined
Information need	To evaluate whether information systems handling sensitive data (confidentiality, integrity) are vulnerable to malicious attacks
Measure	Percentage of critical information systems where a penetration test or vulnerability assessment has been executed since their last major release
Formula/scoring	[Number of information systems quantified as critical and where a penetration test or vulnerability assessment has been done since their last major release/ Number of information systems quantified as critical] * 100, e.g. Green: 100%, Orange >=75%, Red <75%
Target	Orange (Green would be too perfect)
Implementation evidence	Reports of penetration tests or vulnerability assessments performed on information systems compared to number of information systems classified as critical in the asset inventory
Frequency	Collect: yearly Report: for each collection
Responsible parties	Information owner: Risk owner Information collector: Experts with the know-how to conduct penetration tests or execute vulnerability assessments Measurement client: Chief information security officer
Data source	Asset inventory, penetration test reports
Reporting format	Pie chart for current situation and line chart for compliance evolution representation

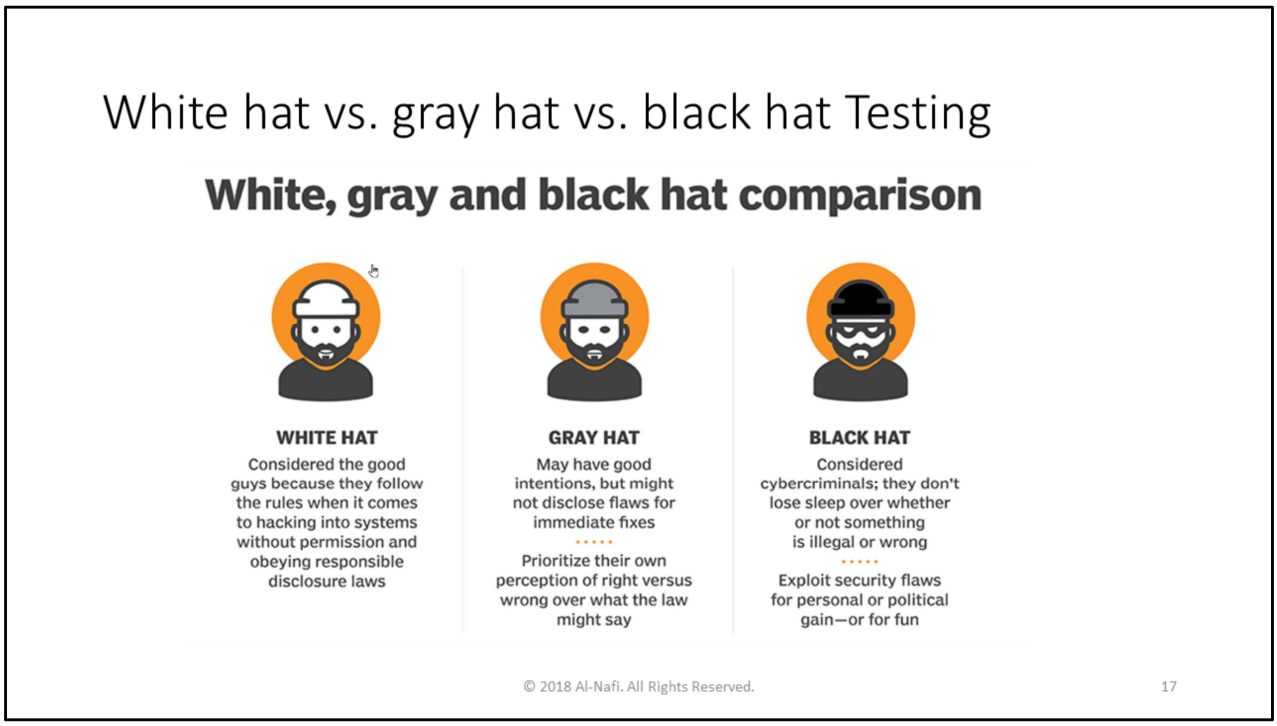
Relationship	ISO/IEC 27001:2013, A.12.6.1: Management of technical vulnerabilities ISO/IEC 27001:2013, A.18.2.3: Technical compliance review

© 2018 Al-Nafi. All Rights Reserved.

Vulnerability assessment vs. Pentesting



© 2018 Al-Nafi. All Rights Reserved.



White hat or Overt Testing

The term "**white hat**" in Internet slang refers to an ethical computer hacker, or a computer security expert, who specializes in penetration testing and in other testing methodologies that ensures the security of an organization's information systems.[1] Ethical hacking is a term meant to imply a broader category than just penetration testing.[2][3] Contrasted with black hat, a malicious hacker, the name comes from Western films, where heroic and antagonistic cowboys might traditionally wear a white and a black hat respectively.[4] While a white hat hacker hacks under good intentions with permission, and a black hat hacker has malicious intent, there is a third kind known as a grey hat hacker who hacks with good intentions without permission.[Symantec Group 1]

White hat hackers may also work in teams called "sneakers",[5] red teams, or tiger teams.[6]

Overt security testing and white hat testing are synonymous terms. Overt testing can be used with both internal and external testing. When used from an internal perspective the bad actor simulated is an employee of the organization. The organization's IT staff is made aware of the testing and can assist the assessor in

limiting the impact of the test by providing specific guidelines for the test scope and parameters. Since overt testing is transparent to the IT staff, it can be an optimal way to train the IT staff. Overt testing carries less risk than covert testing, has lower cost than
covert testing, and is utilized more often than covert testing.

Gray Hat

A **grey hat** (**greyhat** or **gray hat**) is a computer hacker or computer security expert who may sometimes violate laws or typical ethical standards, but does not have the malicious intent typical of a black hat hacker.

The term began to be used in the late 1990s, derived from the concepts of "white hat" and "black hat" hackers.[1] When a white hat hacker discovers a vulnerability, they will exploit it only with permission and not divulge its existence until it has been fixed, whereas the black hat will illegally exploit it and/or tell others how to do so. The grey hat will neither illegally exploit it, nor tell others how to do so.[2]

A further difference among these types of hacker lies in their methods of discovering vulnerabilities. The white hat breaks into systems and networks at the request of their employer or with explicit permission for the purpose of determining how secure it is against hackers, whereas the black hat will break into any system or network in order to uncover sensitive information and for personal gain. The grey hat generally has the skills and intent of the white hat but will break into any system or network without permission.[3][4]

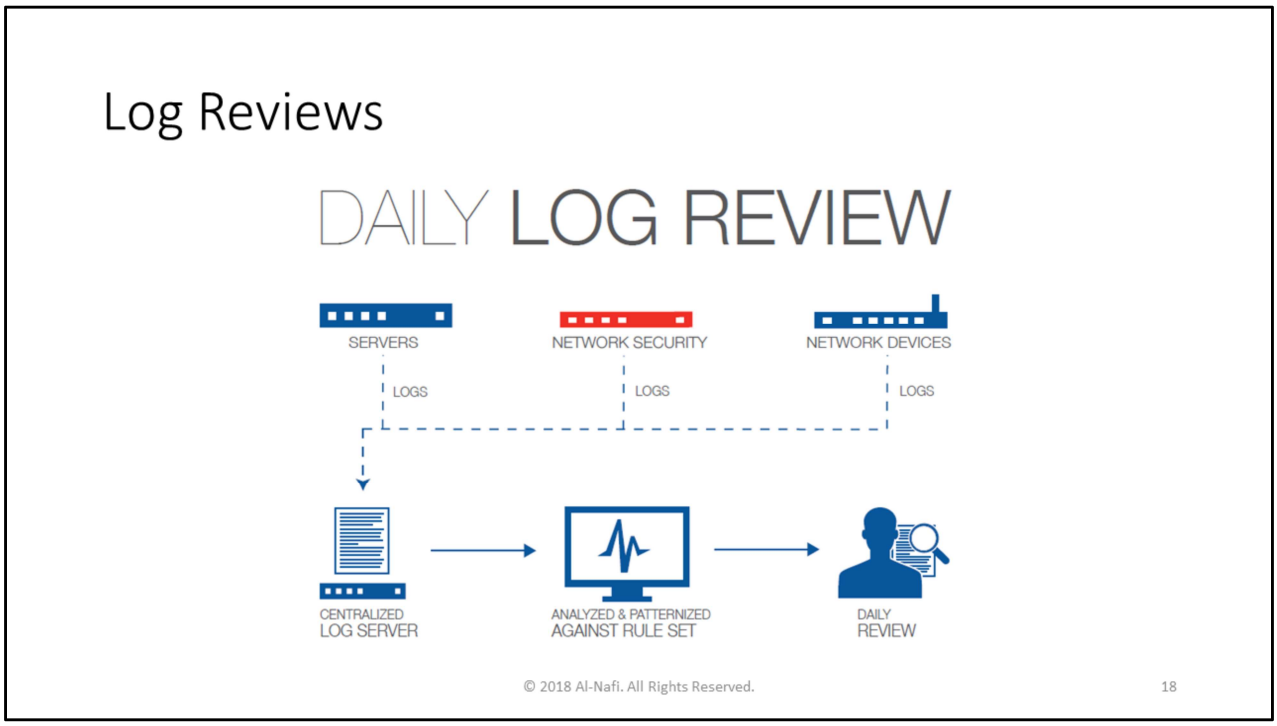
According to one definition of a grey-hat hacker, when they discover a vulnerability, instead of telling the vendor how the exploit works, they may offer to repair it for a small fee. When one successfully gains illegal access to a system or network, they may suggest to the system administrator that one of their friends be hired to fix the problem; however, this practice has been declining due to the increasing willingness of businesses to prosecute. Another definition of Grey hat maintains that Grey hat hackers only arguably violate the law in an effort to research and improve security: legality being set according to the particular ramifications of any hacks they participate in.[5]

Black Hat

The term's origin is often attributed to hacker culture theorist Richard Stallman (though he denies coining it)[1] to contrast the exploitative hacker with the white hat hacker who hacks protectively by drawing attention to vulnerabilities in

computer systems that require repair.[2] The black hat/white hat terminology originates in the Western genre of popular American culture, in which black and white hats denote villainous and heroic cowboys respectively.[3]
Black hat hackers are the stereotypical illegal hacking groups often portrayed in popular culture, and are "the epitome of all that the public fears in a computer criminal".[4] Black hat hackers break into secure networks to destroy, modify, or steal data, or to make the networks unusable for authorized network users.[5]

Covert security testing and black hat testing are synonymous terms. Covert testing is performed to simulate the threats that are associated with external adversaries. While the security staff has no knowledge of the covert test, the organization management is fully aware and consents to the test. A third-party organization may participate in the test as a mitigation point for the security staff's reaction and a communication focal point between the assessors, management, and the security staff. Covert testing will illuminate security staff responsiveness. Typically, the most basic and fundamental exploits are executed within predetermined boundaries and scope to reduce the potential impact of system degradation or damage. Covert tests are often carried out in a stealth fashion, "under the radar," or "slow and low" to simulate an adversary that is seeking to avoid detection. Covert testing provides a comprehensive view of the behavior, posture, and responsiveness of the security staff.



Log Overview
Purpose:
How to detect suspicious activities as soon as possible to reduce the impact of incidence or make prevention if possible.
How to unify the log format and elements as well as the functions?

Role:
Who typically does this?
Security Administrator or independent party who has no access rights/accounts in the reviewed systems. You can't be an user administrator. At the same time, you review your activity everyday. However, if there is a resource limitation, you need another supervisor to authorize your log review.

Frequency:
It depends on the criticality (i.e. payment system, customer information, business secret, etc.) of the system labelled by the organization, logs could be reviewed ranging from minute, every day, weekly, monthly or even 3 months. In fact, log review is a kind of detective control and the preventive control is lacking. Log review

will be the Goal Keeper and frequency is critical.
However, user account and authority list should be reviewed at least 3 to 6 months and never take a check ONLY when the audit cycle is coming

Log Review Tips
Critical systems require at least daily log review, however, what types of logs/activities should we pay attention to?
1. Consecutive login failure especially in non-office hour.
2. Login in non-office hour.
3. Authority change, addition and removal. Check them against with authorized application.
4. Any system administrator's activities
5. Any unknown workstation/server are plugged into the network?
6. Logs removal/log overwritten/log size is full
7. Pay more attention to the log reports after week-end and holiday
8. Any account unlocked/password reset by system administrators without authorized forms?

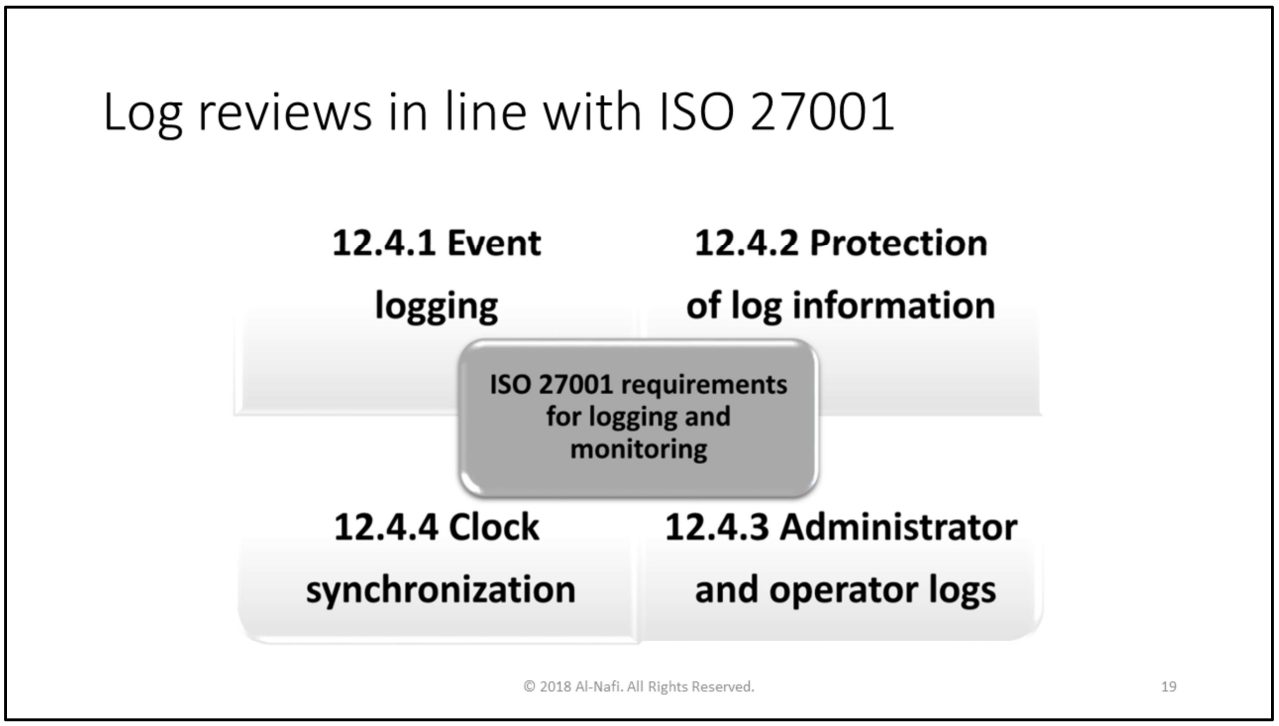
Log Standard
In fact, we are suffering various log format and standard from various systems even we are working in-house or act as a consultant. Why don't we produce a standard/guidelines to developer before they design the user administrative and audit trail functions to fulfill security control.

Functions:-
Search - By date and time, by event type, by criticality, by account/user ID, by department
Sorting - By date and time, by event type, by criticality, by account/user ID, by department
Paging (Optional)
Critical event is marked by "*"
Log archive and export
Log code and description table
Highlighting system and user adminisitrator activities

Mandatory Fields:-
User ID and Name (Sometimes, event may involve the action from administrator)
Activity Date/Timestamp
Activity Code, Type and Description
Terminal IP address and Location

User Account List:-

User Info - Name, Department, Role
Last Accessed Time
Account Creation Date/Time
Current Authority and Role
Account authority and information change history
Show expired and inactive accounts (for example: 90 days)



Logging and monitoring
Objective: To record events and generate evidence.

12.4.1 Event logging
Control
Event logs recording user activities, exceptions, faults and information security events should be
produced, kept and regularly reviewed.
Implementation guidance

Event logs should include, when relevant:
a) user IDs;
b) system activities;
c) dates, times and details of key events, e.g. log-on and log-off;
d) device identity or location if possible and system identifier;
e) records of successful and rejected system access attempts;
f) records of successful and rejected data and other resource access attempts;
g) changes to system configuration;

h) use of privileges;
i) use of system utilities and applications;
j) files accessed and the kind of access;
k) network addresses and protocols;
l) alarms raised by the access control system;
m) activation and de-activation of protection systems, such as anti-virus systems and intrusion detection systems;
n) records of transactions executed by users in applications. Event logging sets the foundation for automated monitoring systems which are capable of generating consolidated reports and alerts on system security.

Other information
Event logs can contain sensitive data and personally identifiable information. Appropriate privacy protection measures should be taken (see 18.1.4).
Where possible, system administrators should not have permission to erase or de-activate logs of their own activities (see 12.4.3).

12.4.2 Protection of log information
Control
Logging facilities and log information should be protected against tampering and unauthorized access.

Implementation guidance
Controls should aim to protect against unauthorized changes to log information and operational problems with the logging facility including:
a) alterations to the message types that are recorded;
b) log files being edited or deleted;
c) storage capacity of the log file media being exceeded, resulting in either the failure to record events or over-writing of past recorded events.
Some audit logs may be required to be archived as part of the record retention policy or because of requirements to collect and retain evidence (see 16.1.7).

Other information

System logs often contain a large volume of information, much of which is extraneous to information security monitoring. To help identify significant events for information security monitoring purposes, the copying of appropriate message types automatically to a second log, or the use of suitable system utilities or audit tools to perform file interrogation and rationalization should be considered. System logs need to be protected, because if the data can be modified or data in them deleted, their existence may create a false sense of security. Real-time copying of logs to a system

outside the control of a system administrator or operator can be used to safeguard logs.

12.4.3 Administrator and operator logs

Control

System administrator and system operator activities should be logged and the logs protected and regularly reviewed.

Implementation guidance

Privileged user account holders may be able to manipulate the logs on information processing facilities under their direct control, therefore it is necessary to protect and review the logs to maintain accountability for the privileged users.

Other information

An intrusion detection system managed outside of the control of system and network administrators can be used to monitor system and network administration activities for compliance.

12.4.4 Clock synchronisation

Control

The clocks of all relevant information processing systems within an organization or security domain should be synchronised to a single reference time source.

Implementation guidance

External and internal requirements for time representation, synchronisation and accuracy should be documented. Such requirements can be legal, regulatory, contractual requirements, standards
compliance or requirements for internal monitoring. A standard reference time for use within the organization should be defined. The organization's approach to obtaining a reference time from external source(s) and how to synchronise internal clocks reliably should be documented and implemented.

Other information

The correct setting of computer clocks is important to ensure the accuracy of audit logs, which may be required for investigations or as evidence in legal or disciplinary cases. Inaccurate audit logs may hinder such investigations and damage the credibility of such evidence. A clock linked to a radio time broadcast from a national atomic clock can be used as the master clock for logging systems. A network time protocol can be used to keep all of the servers in synchronisation with the master clock.

Also read NIST SP 800-92 identifies log reviews section.

Logs everywhere

- **PCI DSS**
- GLBA
- Sarbanes Oxley
- **ISO 27001**
- HIPAA
- GDPR
- You name any regulations they all require log reviews and their protection.



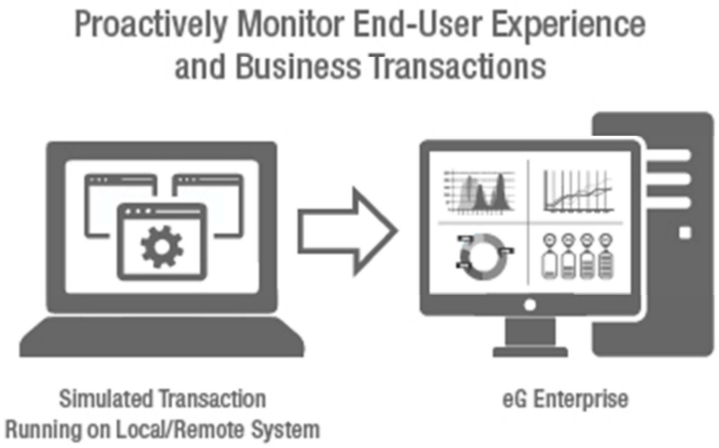
© 2018 Al-Nafi. All Rights Reserved.

Logs Security

- Print and read this document https://www.pcisecuritystandards.org/documents/Effective-Daily-Log-Monitoring-Guidance.pdf
- This will be covered in detail during ISO 27001 and PCI DSS courses.

© 2018 Al-Nafi. All Rights Reserved.

Synthetic Transactions



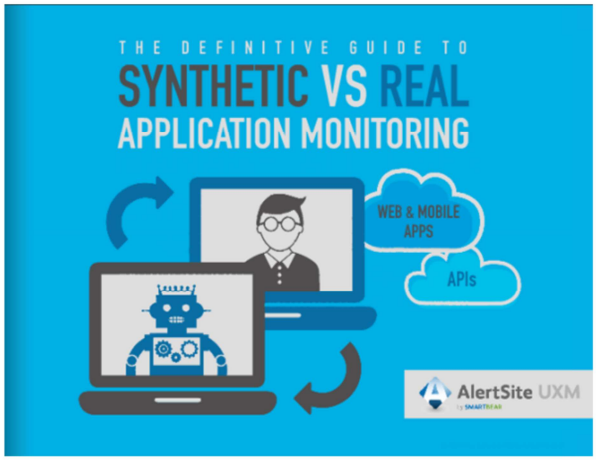
© 2018 Al-Nafi. All Rights Reserved.

"**Synthetic** monitoring (also known as active monitoring or proactive monitoring) is website monitoring that is done using a Web browser emulation or scripted recordings of Web **transactions**. Behavioral scripts (or paths) are created to simulate an action or path that a customer or end-user would take on a site."

It has two types

- Real User Monitoring (RUM)
- Synthetic Performance Monitoring

Synthetic monitoring



© 2018 Al-Nafi. All Rights Reserved.

Synthetic monitoring (also known as ***active monitoring*** *or* ***proactive monitoring***) is a monitoring technique that is done by using an emulation or scripted recordings of transactions. Behavioral scripts (or paths) are created to simulate an action or path that a customer or end-user would take on a site, application or other software (or even hardware). Those paths are then continuously monitored at specified intervals for performance, such as: functionality, availability, and response time measures. Synthetic monitoring is valuable because it enables a webmaster or an IT/Operations professional to identify problems and determine if a website or application is slow or experiencing downtime before that problem affects actual end-users or customers. This type of monitoring does not require actual traffic, thus the name synthetic, so it enables companies to test applications 24x7, or test new applications prior to a live customer-facing launch. This is usually a good complement when used with passive monitoring to help provide visibility on application health during off peak hours when transaction volume is low.[1]
When combined with traditional APM tools, synthetic monitoring can provide deeper visibility into end-to-end performance, regardless of where applications are running.[2]
Because synthetic monitoring is a simulation of typical user behavior or navigation

through a website, it is often best used to monitor commonly trafficked paths and critical business processes. Synthetic tests must be scripted in advance, so it is not feasible to measure performance for every permutation of a navigational path an end-user might take. This is more suited for passive monitoring.
Synthetic testing is useful for measuring uptime, availability and response time of critical pages and transaction (how a site performs from all geographies) but doesn't monitor or capture actual end-user interactions, see Website monitoring. This is also known as Active monitoring that consists of synthetic probes and Web robots to help report on system availability and predefined business transactions.[3]

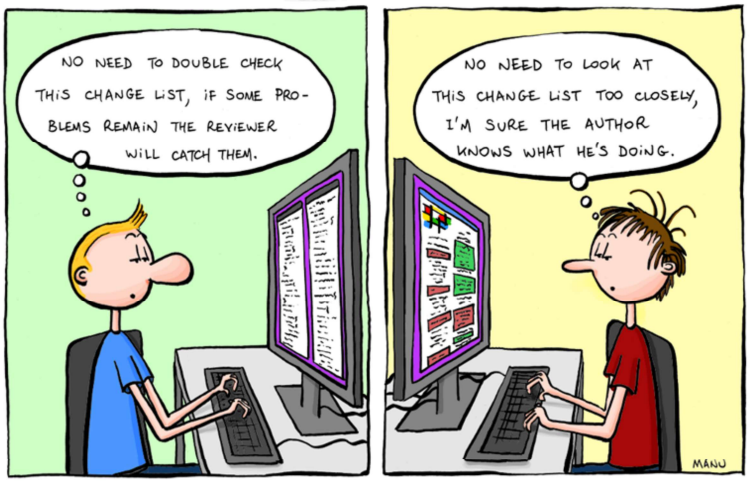
Source : https://en.wikipedia.org/wiki/Synthetic_monitoring

Types of Monitoring
Website Monitoring: Website monitoring uses synthetic transactions to perform HTTP requests to check availability and to measure performance of a web page, website, or web application.

Database Monitoring: Database monitoring using synthetic transactions monitors the availability of a database.

TCP Port Monitoring: A TCP port synthetic transaction measures the availability of your website, service, or application; you can specify the server and TCP port for Operations Manager to monitor.

Code Review and Testing



© 2018 Al-Nafi. All Rights Reserved.

As per ISO 27002:2013 Standard

14.2 Security in development and support processes
Objective: To ensure that information security is designed and implemented within the development lifecycle of information systems.

14.2.1 Secure development policy
Control
Rules for the development of software and systems should be established and applied to developments within the organization.

Implementation guidance
Secure development is a requirement to build up a secure service, architecture, software and system. Within a secure development policy, the following aspects should be put under consideration:
a) security of the development environment;
b) guidance on the security in the software development lifecycle:
    1) security in the software development methodology;

2) secure coding guidelines for each programming language used;
c) security requirements in the design phase;
d) security checkpoints within the project milestones;
e) secure repositories;
f) security in the version control;
g) required application security knowledge;
h) developers' capability of avoiding, finding and fixing vulnerabilities.

Secure programming techniques should be used both for new developments and in code re-use scenarios where the standards applied to development may not be known or were not consistent with current best practices. Secure coding standards should be considered and where relevant mandated for use. Developers should be trained in their use and testing and code review should verify their use. If development is outsourced, the organization should obtain assurance that the external party complies with these rules for secure development.

Other information
Development may also take place inside applications, such as office applications, scripting, browsers and databases.

During Planning and Design
While a security review of the architecture and threat modeling are not security testing methods, they are an important prerequisite for subsequent security testing efforts, and the security practitioner should be aware of the options available to them. The following is a consideration of the prerequisites and benefits of architecture security review and threat modeling:

Architecture security review: A manual review of the product architecture to ensure that it fulfills the necessary security requirements:
o Prerequisites: Architectural model
o Benefit: Detecting architectural violations of the security standard

Threat modeling: A structured manual analysis of an application specific business case or usage scenario. This analysis is guided by a set of precompiled security threats:
o Prerequisites: Business Case or Usage Scenario
o Benefits: Identification of threats, including their impact and potential countermeasures specific to the development of the software product

These methods help to identify the attack surface and, thus, the most critical

components. This allows a focusing of the security testing activities to ensure they are as effective as possible.

What is SAST



© 2018 Al-Nafi. All Rights Reserved.

What is SAST?
Static application security testing (SAST), or static analysis, is a testing methodology that analyzes source code to find security vulnerabilities that make your organization's applications susceptible to attack. SAST scans an application before the code is compiled. It's also known as white box testing.

What problems does SAST solve?
SAST takes place very early in the software development life cycle (SDLC) as it does not require a working application and can take place without code being executed. It helps developers identify vulnerabilities in the initial stages of development and quickly resolve issues without breaking builds or passing on vulnerabilities to the final release of the application. SAST tools give developers real-time feedback as they code, helping them fix issues before they pass the code to the next phase of the SDLC. This prevents security-related issues from being considered an afterthought. SAST tools also provide graphical representations of the issues found, from source to sink. These help you navigate the code easier. Some tools point out the exact location of vulnerabilities and highlight the risky code. Tools can also provide in-depth guidance on how to fix issues and the best place in the code to fix

them, without requiring deep security domain expertise.

Developers can also create the customized reports they need with SAST tools; these reports can be exported offline and tracked using dashboards. Tracking all the security issues reported by the tool in an organized way can help developers remediate these issues promptly and release applications with minimal problems. This process contributes to the creation of a secure SDLC.
It's important to note that SAST tools must be run on the application on a regular basis, such as during daily/monthly builds, every time code is checked in, or during a code release.

Why is SAST an important security activity?
Developers dramatically outnumber security staff. It can be challenging for an organization to find the resources to perform code reviews on even a fraction of its applications. A key strength of SAST tools is the ability to analyze 100% of the codebase. Additionally, they are much faster than manual secure code reviews performed by humans. These tools can scan millions of lines of code in a matter of minutes. SAST tools automatically identify critical vulnerabilities—such as buffer overflows, SQL injection, cross-site scripting, and others—with high confidence. Thus, integrating static analysis into the SDLC can yield dramatic results in the overall quality of the code developed.

What are the key steps to run SAST effectively?
There are six simple steps needed to perform SAST efficiently in organizations that have a very large number of applications built with different languages, frameworks, and platforms.
Finalize the tool. Select a static analysis tool that can perform code reviews of applications written in the programming languages you use. The tool should also be able to comprehend the underlying framework used by your software.
Create the scanning infrastructure, and deploy the tool. This step involves handling the licensing requirements, setting up access control and authorization, and procuring the resources required (e.g., servers and databases) to deploy the tool.
Customize the tool. Fine-tune the tool to suit the needs of the organization. For example, you might configure it to reduce false positives or find additional security vulnerabilities by writing new rules or updating existing ones. Integrate the tool into the build environment, create dashboards for tracking scan results, and build custom reports.
Prioritize and onboard applications. Once the tool is ready, onboard your applications. If you have a large number of applications, prioritize the high-risk

applications to scan first. Eventually, all your applications should be onboarded and scanned regularly, with application scans synced with release cycles, daily or monthly builds, or code check-ins.

Analyze scan results. This step involves triaging the results of the scan to remove false positives. Once the set of issues is finalized, they should be tracked and provided to the deployment teams for proper and timely remediation.

Provide governance and training. Proper governance ensures that your development teams are employing the scanning tools properly. The software security touchpoints should be present within the SDLC. SAST should be incorporated as part of your application development and deployment process.

What tools can be used for SAST?

Synopsys offers the most comprehensive solution for integrating security and quality into your SDLC and supply chain.

Coverity Static Application Security Testing finds critical defects and security weaknesses in code as it's written. It provides full path coverage, ensuring that every line of code and every potential execution path is tested. Through a deep understanding of the source code and the underlying frameworks, it provides highly accurate analysis, so developers don't waste time on a large volume of false positives.

Coverity scales to accommodate thousands of developers and can analyze projects with more than 100 million lines of code with ease. It can be rapidly integrated with critical tools and systems that support the development process, such as source control management, build and continuous integration, bug tracking, and application life cycle management (ALM) solutions, as well as IDEs.

SAST in IDE (Code Sight) is a real-time, developer-centric SAST tool. It scans for and identifies vulnerabilities as developers code. Code Sight integrates into the integrated development environment (IDE), where it identifies security vulnerabilities and provides guidance to remediate them.

How is SAST different from DAST?

Organizations are paying more attention to application security, owing to the rising number of breaches. They want to identify vulnerabilities in their applications and mitigate risks at an early stage. There are two different types of application security testing—SAST and dynamic application security testing (DAST). Both testing methodologies identify security flaws in applications, but they do so differently.

Static source code analysis

- Watch this video https://youtu.be/PF4X2vNv32U

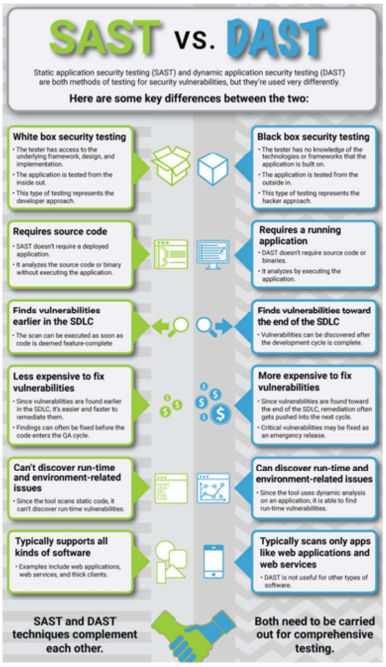
© 2018 Al-Nafi. All Rights Reserved.

Static Source Code Analysis (SAST) and manual code review:
Analysis of the application source code for finding vulnerabilities without executing the application:

- Prerequisites: Application source code
- Benefits: Detection of insecure programming, outdated libraries, and misconfigurations

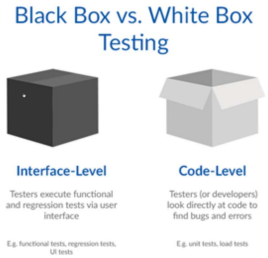
Static binary code analysis and manual binary review: Analysis of the compiled application (binary) for finding vulnerabilities without executing the application. In general, this is like the source
code analysis but is not as precise and fix recommendations typically cannot be provided.

Static Source Code Analysis (SAST) and manual code review



© 2018 Al-Nafi. All Rights Reserved.

Black-box testing vs. white-box testing



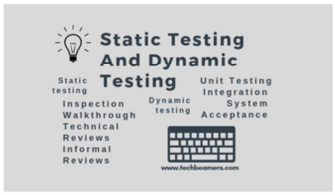
Black Box Testing	White Box Testing
1. Black box testing techniques are also called functional testing techniques.	1. White box testing techniques are also called structural testing techniques.
2Black Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is NOT known to the tester	2. White Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is known to the tester.
3. It is mainly applicable to higher levels of testing such as Acceptance Testing and System Testing	3. Mainly applicable to lower levels of testing such as Unit Testing and Integration Testing
4. Black box testing is generally done by Software Testers	4. White box testing is generally done by Software Developers
5. Programming knowledge is not required	5. Programming knowledge is required
6. Implementation knowledge is not required.	6. Implementation knowledge is required

© 2018 Al-Nafi. All Rights Reserved.

In black-box testing, the tested system is used as a black box, i.e., no internal details of the system implementation are used. In contrast, white-box testing takes the internal system details (e.g., the source code) into account.

Dynamic testing vs. static testing

Static Testing	Dynamic Testing
1. Static Testing is white box testing which is done at early stage of development life cycle. It is more cost effective than dynamic testing	1. Dynamic Testing on the other hand is done at the later stage of development lifecycle.
2. Static Testing Methods include Walkthroughs, code review.	2. Dynamic testing involves functional and nonfunctional testing
3. It is done before code deployment and without execution of code.	3. It is done after code deployment with the execution of codes.
4. This type of testing comes under Verification.	4. This type of testing comes under Validation.
5. Static testing achieves 100% statement coverage in a relatively short time.	5. Dynamic testing may involve running several test cases, each of which may take longer time.
6. Static testing is about prevention.	6. Dynamic testing is about cure.
7. In Static Testing techniques a checklist is prepared for testing process.	7. In Dynamic Testing technique the test cases are executed.



© 2018 Al-Nafi. All Rights Reserved.

Static Testing And Dynamic Testing – Understand the Difference
Software Testing Tutorial | By Meenakshi Agarwal

Static testing and dynamic testing are essential testing techniques meant for developers and testers for use during the Software Development. These are unique validation methods which the organization must decide after due analysis which one to practice for software verification. Since your objective is to get the maximum benefit from these type of testing, hence pick the right tool most suited for your needs. In this tutorial, you'll learn the pros and cons of each of these **type of testing**.

Static Testing And Dynamic Testing
Static Testing:

In Static Testing, it does not require executing the code. You can perform it manually or by using a set of tools. This testing type covers the analysis of the source, review of specification documents, also the design description documents. In this, the testers provide review comments on each of the doc reviewed. If the application is

yet not operational and hasn't implemented the user interface, then you can execute a security analysis to observe it under the runtime-less configuration.

While doing static testing, a tester or a developer can go on to look for the bugs, buffer overflows, and probably identify the vulnerable code in the system. To begin such testing, you don't need to wait for finishing the entire application development. It can get started in an early phase of the development lifecycle. And testers can begin reviewing the code, scripts, requirements, test cases, or any related doc whichever is available at the point of time.

Static Testing Techniques:

1. Inspection:

The principal moto of this Type of Testing is to identify defects in the early stage of the software development cycle. The team can start with the inspection of any of the artifacts as mentioned earlier such the code or the test cases or the product docs walkthroughs. It requires a moderator for organizing the review sessions. Since the inspection is a formal type of review, so the moderator needs to prepare a checklist of what to go through and what not.

2. Walkthrough:

Another technique is the walkthrough. It requires the owner of the document to explain the work done. The attendees can place their queries and another person allocated as the scribe has to
record the points in the notes.

3. Technical Reviews:

In this Static Testing method, the team carries out the Technical Scrutiny of the code written to check whether it meets the coding guidelines and standards. In general, the testing artifacts such as a testing plan, validation strategy, and the automation scripts get also reviewed in this session.

4. Informal Reviews:

Static Testing technique in which the docs get scrutinized informally. Also, the participants provide informal comments during the meeting.

Dynamic Testing:

Dynamic testing gets performed after the application has reached into the operational mode. It gets executed in the real runtime environment set up by the QA. While the code behind the app is running, the tester supplies the required input and waits for the result. After that, he/she matches the output with the expected outcome.

That's how the testers inspect the functional behavior of the application, track the system RAM, CPU usage, its response time, and the performance of the overall software. Dynamic testing has another name as validation testing. It can either be Functional Testing as well as non-functional testing.

Types Of Dynamic Testing Techniques:

1. Unit Testing:
This testing happens mostly at the developer's end. The essential artifacts which get tested are the source code of the application's various modules.

2. Integration Testing:
The purpose of this technique is to verify the interfacing between two or more modules once they get tied up together.

3. System Testing:
This testing gets done on the entire software with all modules working.

4. Acceptance Testing:
This testing runs the validation keeping the user's point of view in mind.

In software development lifecycle, both the Static Testing and Dynamic Testing are essential to certify the application functionality. Each of these has its strength and weakness which you should be aware.

Static Testing Vs. Dynamic Testing:
1. Static Testing belongs to the white box testing. It gets performed at an early stage of development. It incurs a lower cost as compared to the dynamic testing. Vs Dynamic Testing gets performed at the final stage of the development process.
2. Static testing has better line coverage than the dynamic testing in the short duration. Vs Dynamic Testing has lower no. Of line coverage as it examines only a smaller part of the code.
3. Static testing occurs before the application is ready for the deployment. Vs. Dynamic Testing happens after the code got deployed.
4. Static testing gets done in the verification stage. Vs. Dynamic testing completes in the validation stage.
5. No execution happens in the static testing. Vs. Dynamic testing requires the code execution.
6. Static testing produces the analysis of the code along with the documentation. Vs. Dynamic Testing reports the bottlenecks in the application.
7. In Static Testing, the team prepares a checklist describing the testing process. Vs. In

Dynamic Testing, the test cases get executed.
8. Static Testing Methods are Walkthroughs and code reviews. Vs. Dynamic testing majorly has the functional and nonfunctional validation.

Summary – Static Testing And Dynamic Testing
In any Software development methodology, both the Verification and Validation process get carried out to certify that the final software has all the requirements implemented correctly.
Static testing scrutinizes the application code without any execution. It lies under the umbrella of Verification. The testers have got multiple Static testing techniques such as Inspection, Walkthrough, Technical and Informal reviews, etc.

On the contrary, Dynamic testing validates the working product. It lies under the umbrella of Validation. The standard Dynamic testing techniques are Unit Testing, Integration Testing, System or Stabilization Testing and User Acceptance Testing. Here, the product gets validated by both functional and non-functional aspects.

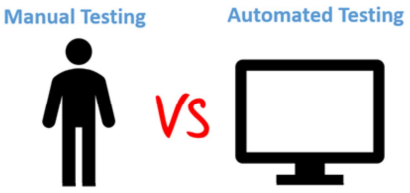
Manual testing vs. automated testing

Manual Testing

- Manual testing is not accurate at all times due to human error, hence it is less reliable.
- Manual testing is time-consuming, taking up human resources.
- Investment is required for human resources.
- Manual testing is only practical when the test cases are run once or twice, and frequent repetition is not required.
- Manual testing allows for human observation, which may be more useful if the goal is user-friendliness or improved customer experience.

Automated Testing

- Automated testing is more reliable, as it is performed by tools and/or scripts.
- Automated testing is executed by software tools, so it is significantly faster than a manual approach.
- Investment is required for testing tools.
- Automated testing is a practical option when the test cases are run repeatedly over a long time period.
- Automated testing does not entail human observation and cannot guarantee user-friendliness or positive customer experience.



© 2018 Al-Nafi. All Rights Reserved.

Manual vs. Automated pentesting



MANUAL PENETRATION TESTING

- In-depth penetration testing
- Performed by security experts often working as external consultants
- Not integrated into the development process

AUTOMATED SECURITY TESTING

- Regular security testing using a fully automated and up-to-date service
- Increases knowledge across the organisation
- Easy-to-understand reports
- Complements compliance and support teams security efforts.



© 2018 Al-Nafi. All Rights Reserved.

Automated vulnerability scanners

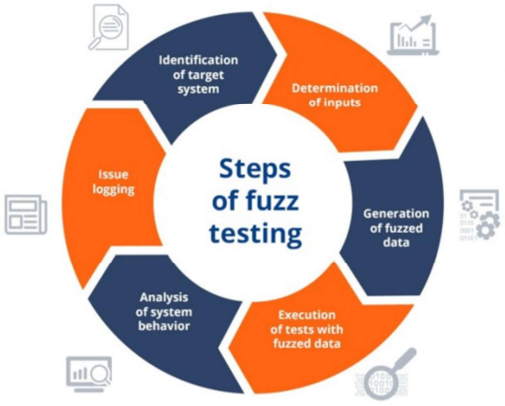


© 2018 Al-Nafi. All Rights Reserved.

Automated vulnerability scanners: Test an application for the use of system components or configurations that are known to be insecure. For this, predefined attack patterns are executed as well as system fingerprints are analyzed:

- o Benefits: Detection of well-known vulnerabilities, i.e., detection of outdated frameworks and misconfigurations

Fuzz testing tools



© 2018 Al-Nafi. All Rights Reserved.

Source Wikipedia:

Fuzzing or **fuzz testing** is an automated software testing technique that involves providing invalid, unexpected, or random data as inputs to a computer program. The program is then monitored for exceptions such as crashes, failing built-in code assertions, or potential memory leaks. Typically, fuzzers are used to test programs that take structured inputs. This structure is specified, e.g., in a file format or protocol and distinguishes valid from invalid input. An effective fuzzer generates semi-valid inputs that are "valid enough" in that they are not directly rejected by the parser, but do create unexpected behaviors deeper in the program and are "invalid enough" to expose corner cases that have not been properly dealt with.

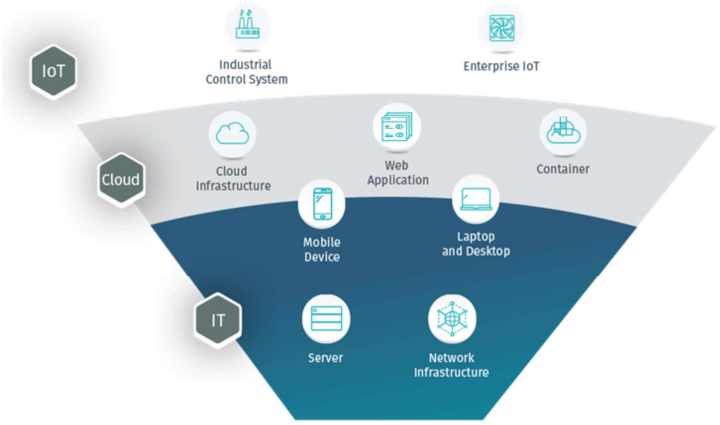
For the purpose of security, input that crosses a trust boundary is often the most interesting.[1] For example, it is more important to fuzz code that handles the upload of a file by any user than it is to fuzz the code that parses a configuration file that is accessible only to a privileged user.

Security testing methods or tools.

- Attack surface
- Application type security testing methods
- Quality of results and usability
- Supported technologies
- Performance and resource utilization

© 2018 Al-Nafi. All Rights Reserved.

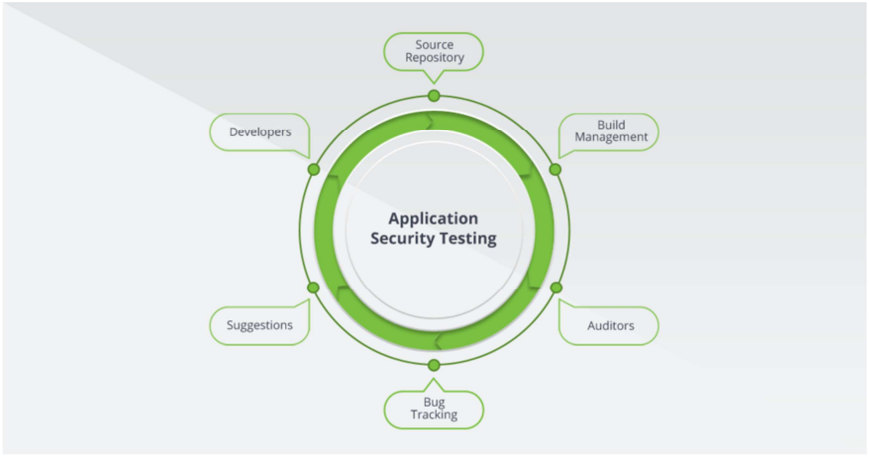
Attack surface



© 2018 Al-Nafi. All Rights Reserved.

The **attack surface** of a software or hardware environment is the sum of the different points (the "**attack** vectors") where an unauthorized user (the "attacker") can try to enter data to or extract data from an environment. Keeping the **attack surface** as small as possible is a basic security measure.

Application type security testing methods



© 2018 Al-Nafi. All Rights Reserved.

Different security testing methods behave differently when applied to different application types

Quality of results and usability



© 2018 Al-Nafi. All Rights Reserved.

Security testing techniques and tools differ in usability (e.g., fix recommendations) and quality (e.g., false positives rate)

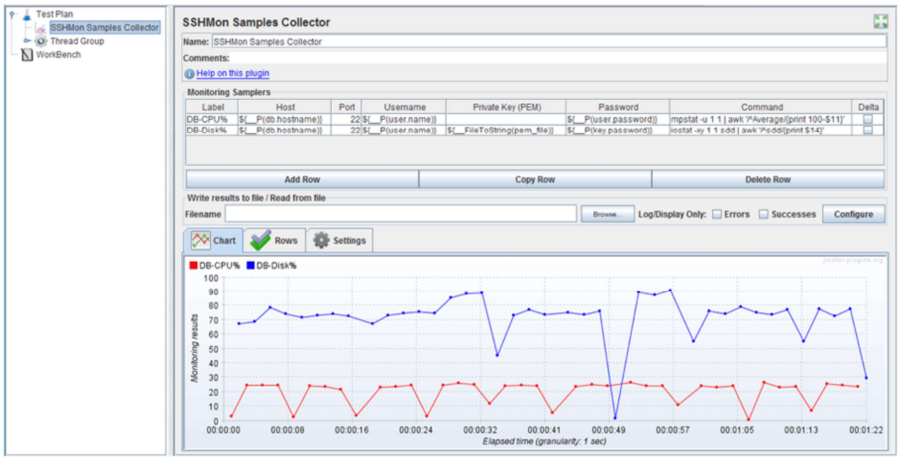
Supported technologies



© 2018 Al-Nafi. All Rights Reserved.

Security testing tools usually only support a limited number of technologies (e.g., programming languages), and if a tool supports multiple technologies, it does not necessarily support all of them equally well

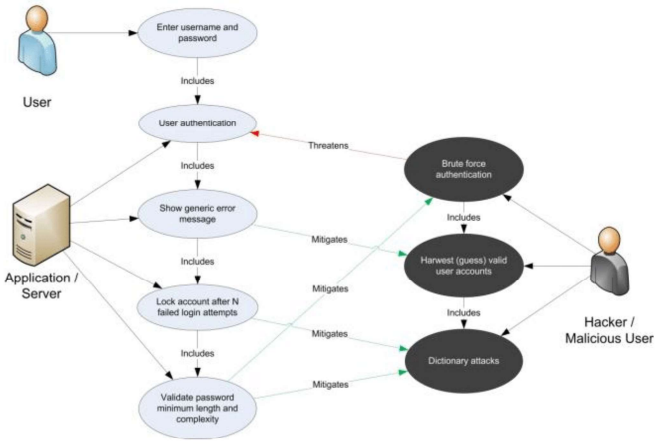
Performance and resource utilization



© 2018 Al-Nafi. All Rights Reserved.

Different tools and methods require different computing power or different manual efforts

Misuse Case Testing

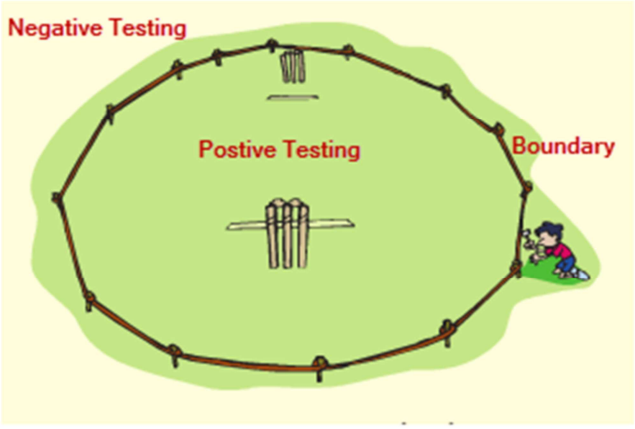


© 2018 Al-Nafi. All Rights Reserved.

Misuse case is a business process modeling tool used in the software development industry. The term *Misuse Case* or *mis-use case* is derived from and is the inverse of use case.[1] The term was first used in the 1990s by Guttorm Sindre of the Norwegian University of Science and Technology, and Andreas L. Opdahl of the University of Bergen, Norway. It describes the process of executing a malicious act against a system, while use case can be used to describe any action taken by the system.[2]

Some misuse cases occur in highly specific situations, whereas others continually threaten systems. For instance, a car is most likely to be stolen when parked and unattended; whereas a web server might suffer a denial-of-service attack at any time. You can develop misuse and use cases recursively, going from system to subsystem levels or lower as necessary. Lower-level cases can highlight aspects not considered at higher levels, possibly forcing another analysis. The approach offers rich possibilities for exploring, understanding, and validating the requirements in any direction. Drawing the agents and misuse cases explicitly helps to focus the attention of the security practitioner on the elements of the scenario.

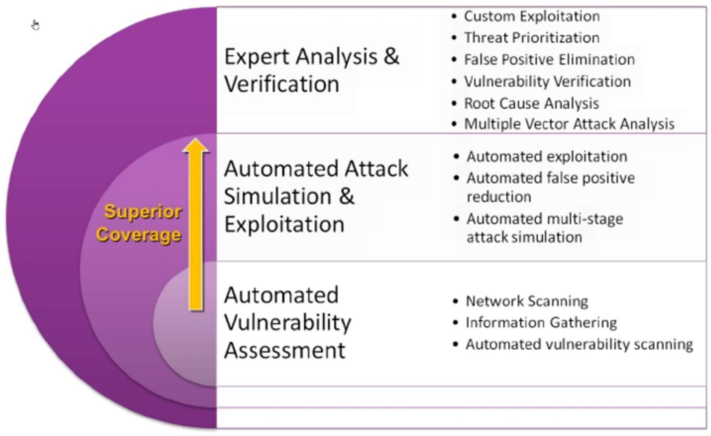
Explore Negative Testing



© 2018 Al-Nafi. All Rights Reserved.

In contrast to a positive test (that determines that a system works as expected, and with any error fails the test); a negative test is designed to provide evidence of the application behavior if there is unexpected or invalid data. Any provocation of application failure is designed to surface in the test rather than once the application is approved for production. An optimal response for an application to a negative test is to gracefully reject the unexpected or invalid data without crashing. While exceptions and error conditions are expected in negative tests
they are not expected in positive tests. It is optimal to combine a range of positive and negative test to run on an application for thorough examination of behavior.

Test-Coverage Analysis

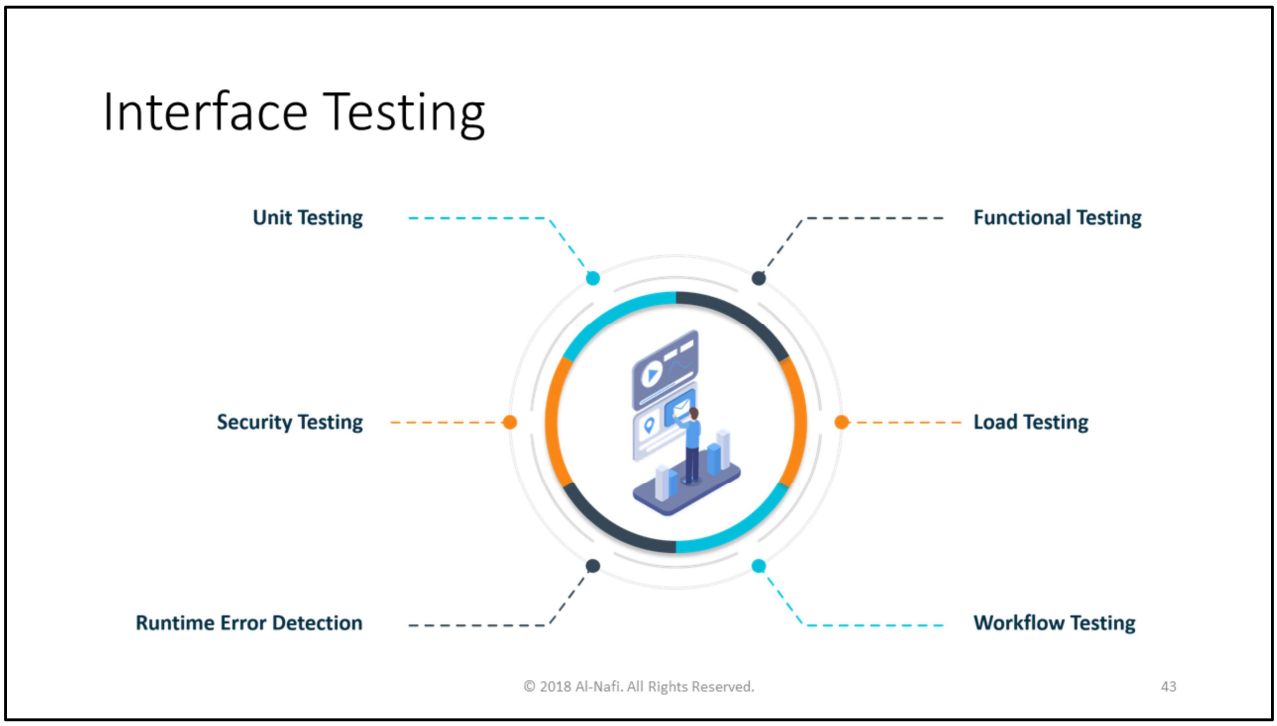


© 2018 Al-Nafi. All Rights Reserved.

Test-Coverage Analyzers

Test-coverage analyzers measure how much of the total program code has been analyzed. The results can be presented in terms of statement coverage (percentage of lines of code tested) or branch coverage (percentage of available paths tested). For large applications, acceptable levels of coverage can be determined in advance and then compared to the results produced by test-coverage analyzers to accelerate the testing-and-release process. These tools can also detect if particular lines of code or branches of logic are not actually able to be reached during program execution, which is inefficient and a potential security concern. Some SAST tools incorporate this functionality into their products, but standalone products also exist.

Since the functionality of analyzing coverage is being incorporated into some of the other AST tool types, standalone coverage analyzers are mainly for niche use.



Interface testing involves the testing of the different components of an application, e.g., software and hardware, in combination. This kind of combination testing is done to ensure they are working correctly and conforming to the requirements based on which they were designed and developed. Interface testing is different from integration testing in that interface testing is done to check whether the different components of the application or system being developed are in sync with each other. In technical terms, interface testing helps determine that distinct functions, such as data transfer between the different elements in the system, are happening according to the way they were designed to happen.
Interface testing is one of the most important software tests in assuring the quality of software products. Interface testing is conducted to evaluate whether systems or components pass data and control correctly to one another. Interface testing is usually performed by both testing and development teams. Interface testing helps to determine which application areas are accessed as well as their user-friendliness.

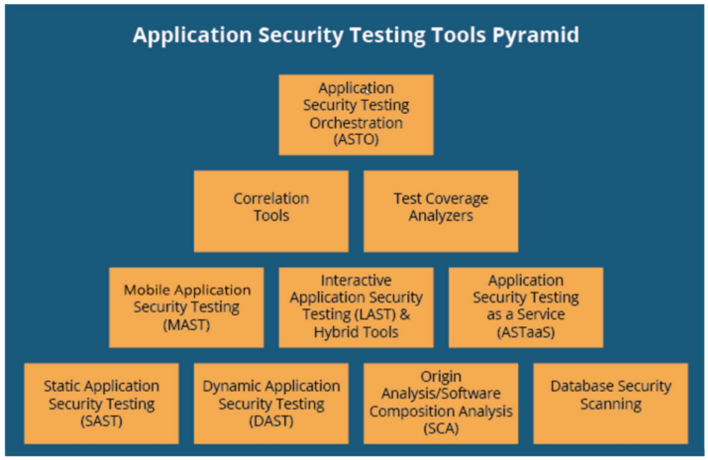
Interface testing can be used to do the following:

- Check and verify if all the interactions between the application and a server are

executed properly

- Check and verify if errors are being handled properly
- Check what happens if a user interrupts any transaction
- Check what happens if a connection to a web server is reset

Application Security Testing tools Pyramid



© 2018 Al-Nafi. All Rights Reserved.

10 Types of Application Security Testing Tools: When and How to Use Them
JULY 9, 2018 • SEI BLOG

By Thomas Scanlon

Bugs and weaknesses in software are common: 84 percent of software breaches exploit vulnerabilities at the application layer. The prevalence of software-related problems is a key motivation for using application security testing (AST) tools. With a growing number of application security testing tools available, it can be confusing for information technology (IT) leaders, developers, and engineers to know which tools address which issues. This blog post, the first in a series on application security testing tools, will help to navigate the sea of offerings by categorizing the different types of AST tools available and providing guidance on how and when to use each class of tool.

See the second post in this series, *Decision-Making Factors for Selecting Application Security Testing Tools*.

Application security is not a simple binary choice, whereby you either have security or you don't. Application security is more of a sliding scale where providing additional security layers helps reduce the risk of an incident, hopefully to an acceptable level of risk for the organization. Thus, application-security testing reduces risk in applications, but cannot completely eliminate it. Steps can be taken, however, to remove those risks that are easiest to remove and to harden the software in use.

The major motivation for using AST tools is that manual code reviews and traditional test plans are time consuming, and new vulnerabilities are continually being introduced or discovered. In many domains, there are regulatory and compliance directives that mandate the use of AST tools. Moreover--and perhaps most importantly--individuals and groups intent on compromising systems use tools too, and those charged with protecting those systems must keep pace with their adversaries.

There are many benefits to using AST tools, which increase the speed, efficiency, and coverage paths for testing applications. The tests they conduct are repeatable and scale well--once a test case is developed in a tool, it can be executed against many lines of code with little incremental cost. AST tools are effective at finding known vulnerabilities, issues, and weaknesses, and they enable users to triage and classify their findings. They can also be used in the remediation workflow, particularly in verification, and they can be used to correlate and identify trends and patterns.

Guide to Application Security Testing Tools
This graphic depicts classes or categories of application security testing tools. The boundaries are blurred at times, as particular products can perform elements of multiple categories, but these are roughly the classes of tools within this domain. There is a rough hierarchy in that the tools at the bottom of the pyramid are foundational and as proficiency is gained with them, organizations may look to use some of the more progressive methods higher in the pyramid.

Static Application Security Testing (SAST)
SAST tools can be thought of as white-hat or white-box testing, where the tester knows information about the system or software being tested, including an architecture diagram, access to source code, etc. SAST tools examine source code (at rest) to detect and report weaknesses that can lead to security vulnerabilities. Source-code analyzers can run on non-compiled code to check for defects such as numerical errors, input validation, race conditions, path traversals, pointers and references, and more. Binary and byte-code analyzers do the same on built and compiled code. Some tools run on source code only, some on compiled code only, and some on both.

Dynamic Application Security Testing (DAST)
In contrast to SAST tools, DAST tools can be thought of as black-hat or black-box testing, where the tester has no prior knowledge of the system. They detect conditions that indicate a security vulnerability in an application in its running state. DAST tools run on operating code to detect issues with interfaces, requests, responses, scripting (i.e., JavaScript), data injection, sessions, authentication, and more.
DAST tools employ fuzzing: throwing known invalid and unexpected test cases at an application, often in large volume.

Origin Analysis/Software Composition Analysis (SCA)
Software-governance processes that depend on manual inspection are prone to failure. SCA tools examine software to determine the origins of all components and libraries within the software. These tools are highly effective at identifying and finding vulnerabilities in common and popular components, particularly open-source components. They do not, however, detect vulnerabilities for in-house custom developed components.

SCA tools are most effective in finding common and popular libraries and components, particularly open-source pieces. They work by comparing known modules found in code to a list of known vulnerabilities. The SCA tools find components that have known and documented vulnerabilities and will often advise if components are out of date or have patches available.
To make this comparison, almost all SCA tools use the NIST National Vulnerability Database Common Vulnerabilities and Exposures (CVEs) as a source for known vulnerabilities. Many commercial SCA products also use the VulnDB commercial vulnerability database as a source, as well as some other public and proprietary sources. SCA tools can run on source code, byte code, binary code, or some combination.

Database Security Scanning
The SQL Slammer worm of 2003 exploited a known vulnerability in a database-management system that had a patch released more than one year before the attack. Although databases are not always considered part of an application, application developers often rely heavily on the database, and applications can often heavily affect databases. Database-security-scanning tools check for updated patches and versions, weak passwords, configuration errors, access control list (ACL) issues, and more. Some tools can mine logs looking for irregular patterns or actions, such as excessive administrative actions.

Database scanners generally run on the static data that is at rest while the database-management system is operating. Some scanners can monitor data that is in transit.

Interactive Application Security Testing (IAST) and Hybrid Tools

Hybrid approaches have been available for a long time, but more recently have been categorized and discussed using the term IAST. IAST tools use a combination of static and dynamic analysis techniques. They can test whether known vulnerabilities in code are actually exploitable in the running application.

IAST tools use knowledge of application flow and data flow to create advanced attack scenarios and use dynamic analysis results recursively: as a dynamic scan is being performed, the tool will learn things about the application based on how it responds to test cases. Some tools will use this knowledge to create additional test cases, which then could yield more knowledge for more test cases and so on. IAST tools are adept at reducing the number of false positives, and work well in Agile and DevOps environments where traditional stand-alone DAST and SAST tools can be too time intensive for the development cycle.

Mobile Application Security Testing (MAST)

The Open Web Application Security Project (OWASP) listed the top 10 mobile risks in 2016 as

improper platform usage
insecure data storage
insecure communication
insecure authentication
insufficient cryptography
insecure authorization
client code quality
code tampering
reverse engineering
extraneous functionality

MAST Tools are a blend of static, dynamic, and forensics analysis. They perform some of the same functions as traditional static and dynamic analyzers but enable mobile code to be run through many of those analyzers as well. MAST tools have specialized features that focus on issues specific to mobile applications, such as jail-breaking or rooting of the device, spoofed WI-FI connections, handling and validation of certificates, prevention of data leakage, and more.

Application Security Testing as a Service (ASTaaS)

As the name suggests, with ASTaaS, you pay someone to perform security testing on your application. The service will usually be a combination of static and dynamic analysis, penetration testing, testing of application programming interfaces (APIs),

risk assessments, and more. ASTaaS can be used on traditional applications, especially mobile and web apps.
Momentum for the use of ASTaaS is coming from use of cloud applications, where resources for testing are easier to marshal. Worldwide spending on public cloud computing is projected to increase from $67B in 2015 to $162B in 2020.

Correlation Tools
Dealing with false positives is a big issue in application security testing. Correlation tools can help reduce some of the noise by providing a central repository for findings from others AST tools.
Different AST tools will have different findings, so correlation tools correlate and analyze results from different AST tools and help with validation and prioritization of findings, including remediation workflows. Whereas some correlation tools include code scanners, they are useful mainly for importing findings from other tools.

Test-Coverage Analyzers
Test-coverage analyzers measure how much of the total program code has been analyzed. The results can be presented in terms of statement coverage (percentage of lines of code tested) or branch coverage (percentage of available paths tested). For large applications, acceptable levels of coverage can be determined in advance and then compared to the results produced by test-coverage analyzers to accelerate the testing-and-release process. These tools can also detect if particular lines of code or branches of logic are not actually able to be reached during program execution, which is inefficient and a potential security concern. Some SAST tools incorporate this functionality into their products, but standalone products also exist.
Since the functionality of analyzing coverage is being incorporated into some of the other AST tool types, standalone coverage analyzers are mainly for niche use.

Application Security Testing Orchestration (ASTO)
ASTO integrates security tooling across a software development lifecycle (SDLC).
While the term ASTO is newly coined by Gartner since this is an emerging field, there are tools that have been doing ASTO already, mainly those created by correlation-tool vendors. The idea of ASTO is to have central, coordinated management and reporting of all the different AST tools running in an ecosystem. It is still too early to know if the term and product lines will endure, but as automated testing becomes more ubiquitous, ASTO does fill a need.

Selecting Testing Tool Types
There are many factors to consider when selecting from among these different types of AST tools. If you are wondering how to begin, the biggest decision you will make is to get started by beginning using the tools. According to a 2013 Microsoft security

study, 76 percent of U.S. developers use no secure application-program process and more than 40 percent of software developers globally said that security wasn't a top priority for them. Our strongest recommendation is that you exclude yourself from these percentages.

There are factors that will help you to decide which type of AST tools to use and to determine which products within an AST tool class to use. It is important to note, however, that no single tool will solve all problems. As stated above, security is not binary; the goal is to reduce risk and exposure.

Before looking at specific AST products, the first step is to determine which type of AST tool is appropriate for your application. Until your application software testing grows in sophistication, most tooling will be done using AST tools from the base of the pyramid, shown in blue in the figure below. These are the most mature AST tools that address most common weaknesses.

After you gain proficiency and experience, you can consider adding some of the second-level approaches shown below in blue. For instance, many testing tools for mobile platforms provide frameworks for you to write custom scripts for testing. Having some experience with traditional DAST tools will allow you to write better test scripts. Likewise, if you have experience with all the classes of tools at the base of the pyramid, you will be better positioned to negotiate the terms and features of an ASTaaS contract.

The decision to employ tools in the top three boxes in the pyramid is dictated as much by management and resource concerns as by technical considerations.

If you are able to implement only one AST tool, here are some guidelines for which type of tool to choose:

If the application is written in-house or you have access to the source code, a good starting point is to run a static application security tool (SAST) and check for coding issues and adherence to coding standards. In fact, SAST is the most common starting point for initial code analysis.

If the application is not written in house or you otherwise don't have access to the source code, dynamic application security testing (DAST) is the best choice.

Whether you have access to the source code or not, if a lot of third-party and open-source components are known to be used in the application, then origin analysis/software composition analysis (SCA) tools are the best choice. Ideally, SCA tools are run alongside SAST and/or DAST tools, but if resources only allow for implementation of one tool, SCA tools are imperative for applications with 3rd party components because they will check for vulnerabilities that are already widely known.

Wrapping Up and Looking Ahead

In the long run, incorporating AST tools into the development process should save time and effort on re-work by catching issues earlier. In practice, however, implementing AST tools requires some initial investment of time and resources. Our guidance presented above is intended to help you select an appropriate starting point. After you begin using AST tools, they can produce lots of results, and someone must manage and act on them.

As you analyze the results with one tool, it may become desirable to introduce additional tools into your environment. As a reference example, the graphic below depicts how many classes of tools could be effectively deployed in a continuous integration and continuous delivery (CI/CD) development process. It is not intended that all these tools be introduced at once into environment. This graphic shows where certain classes of tools fit in to help you make decisions and to provide a roadmap for where you can get to eventually.

These tools also have many knobs and buttons for calibrating the output, but it takes time to set them at a desirable level. Both false positives and false negatives can be troublesome if the tools are not set correctly.

In the next post in this series, I will consider these decision factors in greater detail and present guidance in the form of lists that can easily be scanned and used as checklists by those responsible for application security testing.

Additional Resources

Read the second post in this series, Decision-Making Factors for Selecting Application Security Testing Tools.

Learn about the National Institute of Standards and Technology (NIST) Software Assurance Metrics and Tool Evaluation (SAMATE) Project.

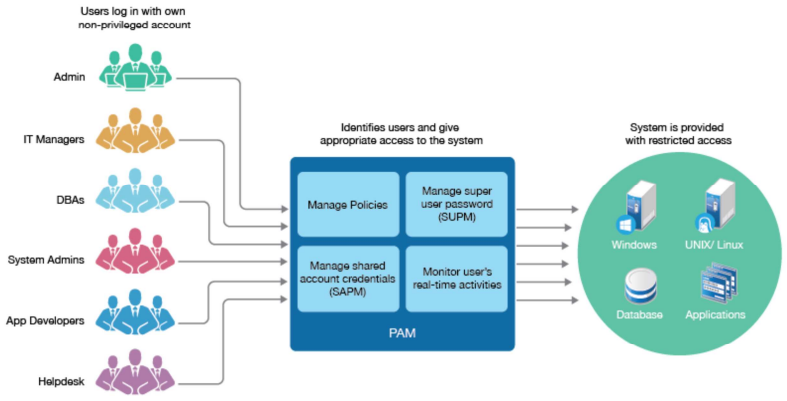
Learn about the Open Web Application Security Project (OWASP).

Learn about the SANS Institute.

Access and download the software, tools, and methods that the SEI creates, tests, refines, and disseminates.

Review the Department of Homeland Security (DHS) Build Security In website.

Account Management Security



© 2018 Al-Nafi. All Rights Reserved.

Account management supports organizational and mission or business functions by:

- Assigning account managers for information systems accounts.
- Establishing conditions for group or role membership.
- Specifying authorized users of information systems.
- Requiring approval for authorizations, creating, enabling, modifying, disabling, and removing access.
- Monitoring use of information systems accounts.
- Notification to account manager when account access is no longer needed.
- Reviews account for compliance with account management requirements.

Management review



© 2018 Al-Nafi. All Rights Reserved.

9.3 Management review

Top management shall review the organization's information security management system at planned intervals to ensure its continuing suitability, adequacy and effectiveness.

The management review shall include consideration of:

a) the status of actions from previous management reviews;

b) changes in external and internal issues that are relevant to the information security management system;

c) feedback on the information security performance, including trends in:

1) nonconformities and corrective actions;
2) monitoring and measurement results;
3) audit results; and
4) fulfilment of information security objectives;

d) feedback from interested parties;

e) results of risk assessment and status of risk treatment plan; and

f) opportunities for continual improvement.

The outputs of the management review shall include decisions related to continual

improvement opportunities and any needs for changes to the information security management system.
The organization shall retain documented information as evidence of the results of management reviews.

Key Performance and Risk Indicators



© 2018 Al-Nafi. All Rights Reserved.

Download this document and read end to end to understand KPI's and KRI's.

https://www.coso.org/Documents/COSO-KRI-Paper-Full-FINAL-for-Web-Posting-Dec110-000.pdf

It is important to distinguish key performance indicators (KPIs) from key risk indicators (KRIs). Both management and boards regularly review summary data that include selected KPIs designed to provide a high-level overview of the performance of the organization and its major operating units. These reports often are focused almost exclusively on the historical performance of the organization and its key units and operations. For example, reports often highlight monthly, quarterly, and year-to-date sales trends, customer shipments, delinquencies, and other performance data points relevant to the organization. It is important to recognize that these measures may not provide an adequate "early warning indicator" of a developing risk because they mostly focus on results that have already occurred.

While KPIs are important to the successful management of an organization by identifying underperforming aspects of the enterprise as well as those aspects of the business that merit increased resources and energy, senior management and boards also benefit from a set of KRIs that provide timely leading-indicator information about emerging risks.

Measures of events or trigger points that might signal issues developing internally within the operations of the organization or potential risks emerging from external events, such as macroeconomic shifts that affect the demand for the organization's products or services, may provide rich information for management and boards to consider as they execute the strategies of the organization.

Key risk indicators are metrics used by organizations to provide an early signal of increasing risk exposures in various areas of the enterprise. In some instances, they may represent key ratios that management throughout the organization track as indicators of evolving risks, and potential opportunities, which signal the need for actions that need to be taken. Others may be more elaborate and involve the aggregation of several individual risk indicators into a multi-dimensional score about emerging events that may lead to new risks or opportunities. An example related to the oversight of accounts receivable collection helps illustrate the difference in KPIs and KRIs. A key performance indicator for customer credit is likely to include data about customer delinquencies and write-offs. This key performance indicator, while important, provides insights about a risk event that has already occurred (e.g., a customer failed to pay in accordance with the sales agreement or contract). A KRI could be developed to help anticipate potential future customer collection issues so that the credit function could be more proactive in addressing customer payment trends before risk events occur. A relevant KRI for this example might be analysis of reported financial results of the company's 25 largest customers or general collection challenges throughout the industry to see what trends might be emerging among customers that could potentially signal challenges related to collection efforts in future periods.

Developing Effective Key Risk Indicators A goal of developing an effective set of KRIs is to identify relevant metrics that provide useful insights about potential risks that may have an impact on the achievement of the organization's objectives. Therefore, the selection and design of effective KRIs starts with a firm grasp of organizational objectives and risk-related events that might affect the achievement of those objectives. Linkage of top risks to core strategies helps pinpoint the most relevant information that might serve as an effective leading indicator of an emerging risk. In the simple illustration below, management has an objective to achieve greater profitability by increasing revenues and decreasing costs. They have identified four strategic initiatives that are critical to accomplishing those objectives. Several potential risks have been identified that may have an impact on one or more of four key strategic initiatives. Mapping key risks to core strategic initiatives puts management in a position to begin identifying the most critical metrics that can serve as leading key risk indicators to help them oversee the execution of core strategic initiatives. As shown below, KRIs have been identified for each critical risk. Mapping KRIs to critical risks and core strategies reduces the likelihood that management becomes distracted by other information that may be less relevant to the achievement of enterprise objectives.

Data Backup and Verification



© 2018 Al-Nafi. All Rights Reserved.

12.3 Backup

Objective: To protect against loss of data.

12.3.1 Information backup

Control

Backup copies of information, software and system images should be taken and tested regularly in accordance with an agreed backup policy.

Implementation guidance

A backup policy should be established to define the organization's requirements for backup of information, software and systems. The backup policy should define the retention and protection requirements.

Adequate backup facilities should be provided to ensure that all essential information and software can be recovered following a disaster or media failure.

When designing a backup plan, the following items should be taken into consideration:

a) accurate and complete records of the backup copies and documented restoration procedures should be produced;

b) the extent (e.g. full or differential backup) and frequency of backups should reflect the business requirements of the organization, the security requirements of the information involved and the
criticality of the information to the continued operation of the organization;
c) the backups should be stored in a remote location, at a sufficient distance to escape any damage from a disaster at the main site;
d) backup information should be given an appropriate level of physical and environmental protection (see Clause 11) consistent with the standards applied at the main site;
e) backup media should be regularly tested to ensure that they can be relied upon for emergency use when necessary; this should be combined with a test of the restoration procedures and checked against the restoration time required. Testing the ability to restore backed-up data should be performed onto dedicated test media, not by overwriting the original media in case the backup or restoration process fails and causes irreparable data damage or loss;
f) in situations where confidentiality is of importance, backups should be protected by means of encryption. Operational procedures should monitor the execution of backups and address failures of scheduled backups to ensure completeness of backups according to the backup policy. Backup arrangements for individual systems and services should be regularly tested to ensure that
they meet the requirements of business continuity plans. In the case of critical systems and services, backup arrangements should cover all systems information, applications and data necessary to recover the complete system in the event of a disaster. The retention period for essential business information should be determined, taking into account any requirement for archive copies to be permanently retained.

Training and Awareness



© 2018 Al-Nafi. All Rights Reserved.

Training is defined in NIST Special Publication 800-16 as follows: “The ‘Training’ level of the learning continuum strives to produce relevant and needed security skills and competencies by practitioners of functional specialties other than IT security (e.g., management, systems design and development, acquisition, auditing).” The most significant difference between training and awareness is that training seeks to teach skills, which allow a person to perform a specific function, while awareness seeks to focus an individual’s attention on an issue or set of issues. The skills acquired during training are built upon the awareness foundation, in particular, upon the security basics and literacy material. A training curriculum must not necessarily lead to a formal degree from an institution of higher learning; however, a training course may contain much of the same material found in a course that a college or university includes in a certificate or degree program.

An example of training is an IT security course for system administrators, which should address in detail the management controls, operational controls, and technical controls that should be implemented. Management controls include policy, IT security program management, risk management, and life-cycle security.

Operational controls include personnel and user issues, contingency planning, incident handling, awareness and training, computer support and operations, and physical and environmental security issues. Technical controls include identification and authentication, logical access controls, audit trails, and cryptography. (See NIST Special Publication 800-12, *An Introduction to Computer Security: The NIST Handbook*, for in-depth discussion of these controls (http://csrc.nist.gov/publications/nistpubs/index.html).)

Please download and review
https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-50.pdf

Executive management: Organizational leaders need to fully understand directives and laws that form the basis for the security program. They also need to comprehend their leadership roles in ensuring full compliance by users within their units.

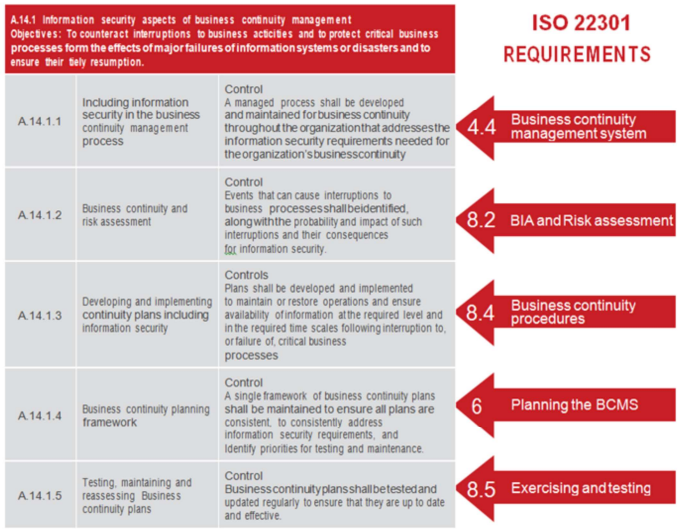
Security personnel (security program managers and security officers): These individuals act as expert consultants for their organization; therefore, they must be well educated on security policy and accepted best practices.

System owners: Owners must have a broad understanding of security policy and a high degree of understanding regarding security controls and requirements applicable to the systems they manage.

System administrators and IT support personnel: Entrusted with a high degree of authority over support operations critical to a successful security program, these individuals need a higher degree of technical knowledge in effective security practices and implementation.

Operational managers and system users: These individuals need a high degree of security awareness and training on security controls and rules of behavior for systems they use to conduct business operations.

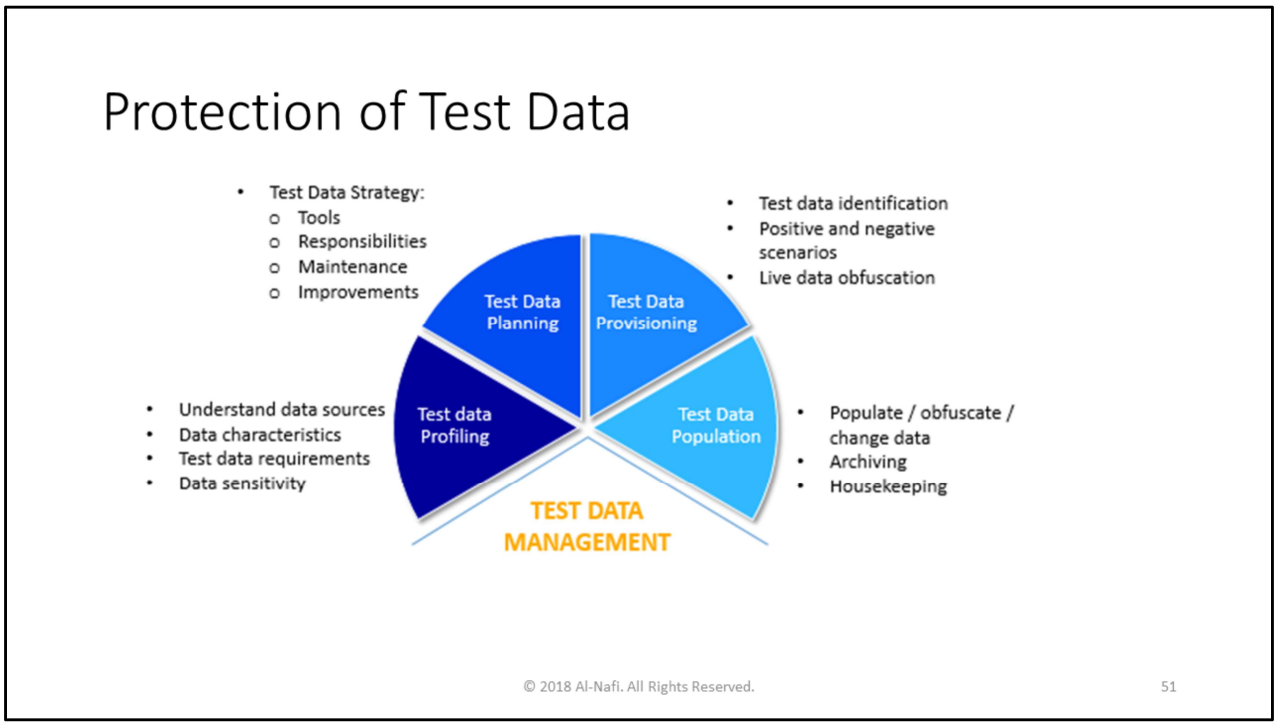
Disaster Recovery (DR) and Business Continuity (BC)



A.14.1 Information security aspects of business continuity management Objectives: To counteract interruptions to business activities and to protect critical business processes form the effects of major failures of information systems or disasters and to ensure their tiely resumption.			ISO 22301 REQUIREMENTS
A.14.1.1	Including information security in the business continuity management process	Control A managed process shall be developed and maintained for business continuity throughout the organization that addresses the information security requirements needed for the organization's business continuity	4.4 Business continuity management system
A.14.1.2	Business continuity and risk assessment	Control Events that can cause interruptions to business processes shall be identified, along with the probability and impact of such interruptions and their consequences for information security.	8.2 BIA and Risk assessment
A.14.1.3	Developing and implementing continuity plans including information security	Controls Plans shall be developed and implemented to maintain or restore operations and ensure availability of information at the required level and in the required time scales following interruption to, or failure of, critical business processes	8.4 Business continuity procedures
A.14.1.4	Business continuity planning framework	Control A single framework of business continuity plans shall be maintained to ensure all plans are consistent, to consistently address information security requirements, and identify priorities for testing and maintenance.	6 Planning the BCMS
A.14.1.5	Testing, maintaining and reassessing Business continuity plans	Control Business continuity plans shall be tested and updated regularly to ensure that they are up to date and effective.	8.5 Exercising and testing

© 2018 Al-Nafi. All Rights Reserved.

We will cover this in detail in later domains as it was covered to some extent in the EARLIER DOMAINS. We will also cover ISO 22301 Lead Implementation Course as well inshAllah.



14.3.1 Protection of test data

Control

Test data should be selected carefully, protected and controlled.

Implementation guidance

The use of operational data containing personally identifiable information or any other confidential information for testing purposes should be avoided. If personally identifiable information or otherwise confidential information is used for testing purposes, all sensitive details and content should be protected by removal or modification (see ISO/IEC 29101[26]).

The following guidelines should be applied to protect operational data, when used for testing purposes:

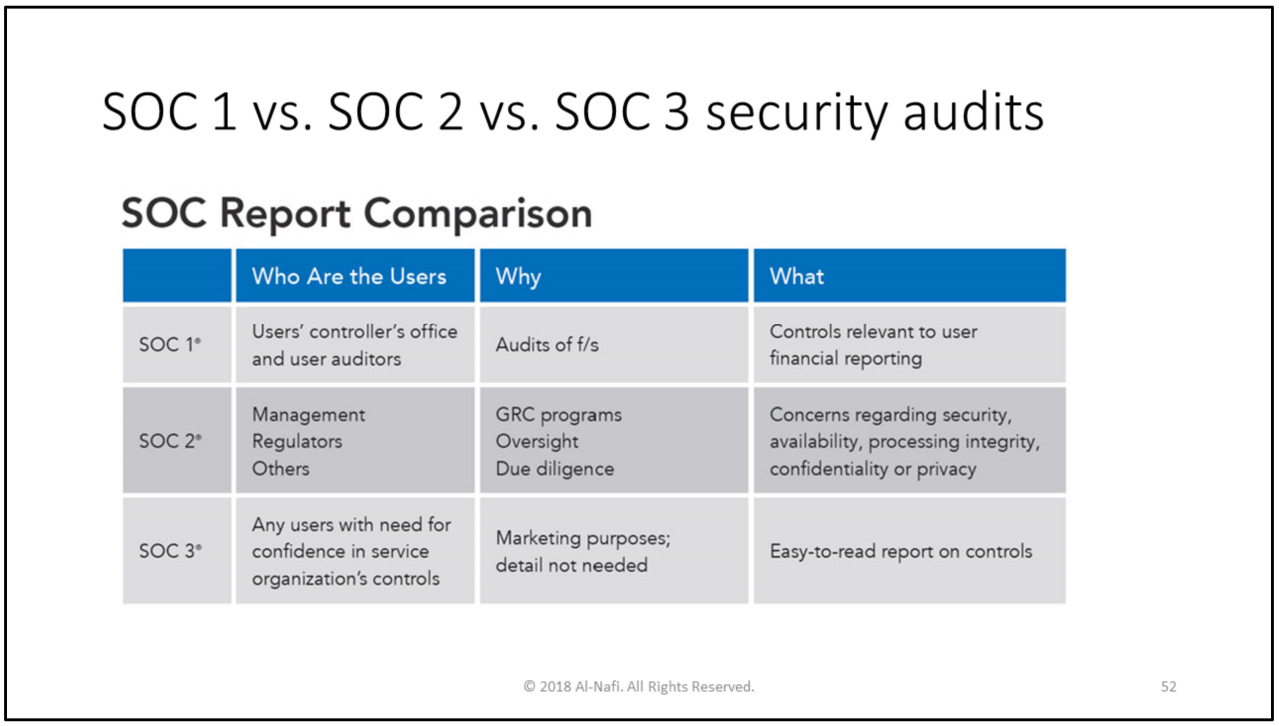
a) the access control procedures, which apply to operational application systems, should also apply to test application systems;

b) there should be separate authorization each time operational information is copied to a test environment;

c) operational information should be erased from a test environment immediately

after the testing is complete;
d) the copying and use of operational information should be logged to provide an audit trail.

Other information
System and acceptance testing usually requires substantial volumes of test data that are as close as possible to operational data.



Review this link. Joseph Kirkpatrick is an old friend of mine and he will be helping us achieve ISO 27001 certification along with SOC1, 2 and 3 down the line once we open our other business etc.

https://kirkpatrickprice.com/video/soc-1-vs-soc-2-vs-soc-3/

What's The Difference Between SOC 1, SOC 2, and SOC 3?
August 16, 2017/by Joseph KirkpatrickWhen it comes to SOC (Service Organization Control) reports, there are three different report types: SOC 1, SOC 2, and SOC 3. When considering which report fits your organization's needs, you must first understand what your clients require of you and then consider the areas of internal control over financial reporting (ICFR), the Trust Services Principles, and restricted use.
SOC 1 vs. SOC 2 vs. SOC 3

What Is a SOC 1 Report?
SOC 1 engagements are based on the SSAE 18 standard and report on the effectiveness of internal controls at a service organization that may be relevant to their client's internal control over financial reporting (ICFR).

What Is a SOC 2 Report?

A SOC 2 audit evaluates internal controls, policies, and procedures that directly relate to the security of a system at a service organization. The SOC 2 report was designed to determine if service organizations are compliant with the principles of security, availability, processing integrity, confidentiality, and privacy, also known as the Trust Services Principles. These principles address internal controls unrelated to ICFR.

What Is a SOC 3 Report?

A SOC 3 report, just like a SOC 2, is based on the Trust Services Principles, but there's a major difference between these types of reports: restricted use. A SOC 3 report can be freely distributed, whereas a SOC 1 or SOC 2 can only be read by the user organizations that rely on your services. A SOC 3 does not give a description of the service organization's system, but can provide interested parties with the auditor's report on whether an entity maintained effective controls over its systems as it relates to the Trust Services Principles.

When trying to determine whether your service organization needs a SOC 1, SOC 2, or SOC 3, keep these requirements in mind:

Could your service organization affect a client's financial reporting? A SOC 1 would apply to you.

Does your service organization want to be evaluated on the Trust Service Principles? SOC 2 and SOC 3 reports would work.

Does restricted use affect your decision? SOC 1 and SOC 2 reports can only be read by the user organizations that rely on your services. A SOC 3 report can be freely distributed, used in many different applications.

Each of these reports must be issued by a licensed CPA firm, such as KirkpatrickPrice. We offer SOC 1, SOC 2, and SOC 3 engagements. To learn more about KirkpatrickPrice's SOC services, contact us today using the form below.

SOC 1 vs. SOC 2 vs. SOC 3 audits

A new course will be released as part of the audit track to provide walk through on how to perform SOC audits as per 1, 2 and 3 requirements. This course will be delivered in July 2020 inshAllah.

© 2018 Al-Nafi. All Rights Reserved.

جزاك اللهُ

To ask questions, Join the Al Nafi Official Group

https://www.facebook.com/groups/alnafi/

(This group is only for members to ask questions)

© 2018 Al-Nafi. All Rights Reserved.